



*Asociación Profesional de Cuerpos Superiores
de Sistemas y Tecnologías de la Información
de las Administraciones Públicas.*

Temas Específicos para la preparación de la Oposición al Cuerpo
Superior de Sistemas y Tecnologías de la Información de la
Administración del Estado.

TEMAS ESPECÍFICOS III: Ingeniería de los Sistemas de Información

**Tema 89. Análisis del dominio de los sistemas: Modelado de
dominio, modelo entidad relación y modelos de clases**

AUTOR: Mariam Dobón Muñoz.

Actualización 2017

ÍNDICE

1	INTRODUCCIÓN	3
2	ANÁLISIS DEL DOMINIO DE LOS SISTEMAS.....	4
2.1	ANÁLISIS Y MODELADO DE SISTEMAS DE INFORMACIÓN	4
2.2	MODELADO DE DOMINIO.....	6
2.3	MODELO ENTIDAD RELACIÓN	8
2.3.1	ELEMENTOS DEL MODELO E/R	9
2.3.1.1	ENTIDADES	9
2.3.2	RELACIONES.....	10
2.3.3	ATRIBUTOS.....	11
2.3.4	RESTRICCIONES	12
2.3.5	MODELO ENTIDAD-RELACIÓN EXTENDIDO.....	13
2.3.5.1	ESPECIALIZACIÓN - GENERALIZACIÓN.....	14
2.3.5.2	CARDINALIDAD MÍNIMA DE UN TIPO DE ENTIDAD	15
2.3.5.3	DEPENDENCIA EN EXISTENCIA Y EN IDENTIFICACIÓN	15
2.3.5.4	AGREGACIÓN	15
2.3.6	TRADUCCIÓN DEL MODELO E/R AL MODELO RELACIONAL	16
2.3.7	VENTAJAS Y DESVENTAJAS DEL MODELO ENTIDAD-RELACIÓN	17
2.4	MODELOS DE CLASES	17
2.4.1	CLASES.....	18
2.4.1.1	RELACIONES ENTRE CLASES.....	23
2.4.1.2	CRITERIOS PARA IDENTIFICAR CLASES EN EL MODELO DE CLASES	26
2.5	OTRAS TÉCNICAS RELACIONADAS	26
2.5.1	DIAGRAMA DE OBJETOS	27
2.5.2	DIAGRAMA DE PAQUETES	28
2.5.3	DIAGRAMA DE ESTRUCTURA COMPUESTA	30
2.5.4	DICCIONARIO DE DATOS.....	32
2.5.4.1	CARACTERÍSTICAS DEL DICCIONARIO DE DATOS	33
2.5.4.2	DATOS QUE SE ALMACENAN EN UN DICCIONARIO DE DATOS	33
2.5.4.3	DEFINICIÓN DEL FORMATO DE LOS DATOS EN EL DICCIONARIO DE DATOS	33
2.5.4.4	OPERACIONES QUE PERMITE UN DICCIONARIO DE DATOS	35
2.5.4.5	TIPOS DE DICCIONARIOS.....	35
3	CONCLUSIÓN	36
4	GLOSARIO	37

1 Introducción

En el tema anterior se ha profundizado en la fase de Análisis de un Sistema de Información desde una perspectiva funcional o dinámica. En el presente tema se continúa elaborando la fase de análisis, pero ya desde el enfoque estático o de modelado de dominio.

Como se ha visto en temas anteriores, existen distintas definiciones para la fase de Análisis del Sistema de Información, pero todas ellas coinciden en el hecho de que el análisis parte de los requisitos que surgen durante el proceso de estudio del dominio del problema y arroja como resultado lo que debe hacer el sistema para encontrar una solución al problema del usuario, independientemente de la naturaleza de la tecnología que se use para su implementación.

Tomando como ejemplo la definición proporcionada en la metodología Métrica v3, el Análisis del Sistema de Información (ASI) tiene por objetivo la obtención de una especificación detallada del sistema que satisfaga las necesidades de información de los usuarios y sirva de base para el posterior diseño del mismo.

De esta manera, el Análisis hace de puente entre el problema y el diseño, sirviendo así de enlace entre la descripción del sistema y el modelo del diseño, identificando las necesidades y restricciones del problema y generando unas especificaciones que sirvan de base para crear un diseño.

Así, el resultado del análisis del sistema es el conjunto de modelos que describen el comportamiento del sistema de información. Estos modelos deben ser lo suficientemente detallados para que en el paso de diseño se pueda construir un sistema capaz de cumplir las necesidades del usuario.

En concreto, el análisis del dominio del sistema trata de identificar y entender los conceptos más importantes del dominio del problema. El análisis del dominio tiene como finalidad principal la captura y adquisición de la información que deben almacenar y procesar las aplicaciones informáticas orientadas a una función o uso. Para ello, ofrece un conjunto de técnicas o herramientas para capturar la información crítica sobre las entidades, datos y procesos característicos de un área de actividad, y así facilitar la especificación, el diseño y la construcción de los sistemas informáticos relacionados. El análisis de dominio, asimismo, pretende la reutilización del conocimiento y la experiencia adquirida durante el diseño y construcción de los sistemas informáticos de un mismo tipo.

En el presente tema se analizan diversas técnicas de análisis del dominio de un sistema de información, profundizando principalmente en el modelo de dominio, el modelo de clases y el modelo entidad relación. No obstante, también se detallan otras técnicas relevantes en el campo que nos ocupa. Es el caso del diagrama de paquetes, el diagrama de estructura compuesta o el diagrama de objetos en programación orientada a objetos e incluidas en UML (Unified Modelling Language), o el diccionario de datos, técnica utilizada principalmente en el análisis estructurado y complementaria a otras técnicas como el Diagramas de Flujos de Datos (DFDs).

2 Análisis del dominio de los sistemas

En este epígrafe se abordan todos los conceptos del análisis de dominio de los sistemas. En primer lugar, se explica qué es el análisis y modelado de un sistema de información para luego entrar a detallar las técnicas de modelado de dominio, entidad relación, el modelo de clases, y el resto de técnicas relacionadas.

2.1 Análisis y modelado de sistemas de información

Durante la fase de análisis de un sistema de información se estudia un sistema o necesidad existente, para bien proponer nuevos sistemas o la inclusión de mejoras en los sistemas actuales. Entre los objetivos de dicha fase, destaca la obtención de una especificación detallada del sistema de información a través del catálogo de requisitos y de diferentes modelos del sistema.

El catálogo de requisitos se obtiene mediante la realización de un análisis de requisitos para reunir información y determinar los requerimientos. Unos requisitos bien entendidos son determinantes para producir software de calidad. Por su parte, la elaboración de los modelos del sistema o modelos de análisis sirve para presentar distintos puntos de vista del sistema: el punto de vista funcional, el estático y el dinámico:

- El **punto de vista funcional** describe cómo se transforman los datos dentro del sistema, cómo se convierten las entradas en salidas.
- El **punto de vista estático** describe los objetos de información del sistema y las relaciones que existen entre ellos.
- El **punto de vista dinámico** describe los elementos del sistema que varían a lo largo del tiempo.

El presente tema, como se verá más adelante, se centra en el punto de vista estático, mientras que los otros dos puntos de vista son cubiertos por otros temas del temario.

Con todo ello, el modelo de análisis debe cumplir tres objetivos:

- Describir lo que el cliente quiere.
- Establecer una base para la creación de un diseño de software, es decir, definir una especificación detallada del funcionamiento del sistema.
- Definir un conjunto de requisitos que se puedan validar cuando esté construido el sistema.

Durante esta fase se genera gran cantidad de información, la cual debe ser analizada, estructurada, integrada, validada y verificada por los diferentes participantes del mismo. Durante la primera parte del análisis, es decir, el análisis del problema, el objetivo que se tiene es el de entender el problema.

Una vez entendidas las necesidades, los analistas proceden a especificar el sistema de una forma clara y no ambigua. La fase de análisis culmina con dicha especificación del producto, en la que sirve de documento de base para todos los participantes. A esta especificación se la suele conocer como ERS o Especificación de Requisitos del Software, y en ella se recogen todos los detalles que debe cumplir el producto para que cubra las necesidades del usuario.

A partir de la ERS, los diseñadores comenzarán con la fase de Diseño del Sistema y construirán un sistema que dé respuesta a las especificaciones indicadas en la ERS.

Una ERS es un documento que define, de forma completa, precisa y verificable, los requisitos, el comportamiento u otras características del sistema a desarrollar o de cualquiera de sus componentes. En la especificación se incluye toda la descripción y documentación de todos los requisitos y su relación con el exterior. Se tiene así que la ERS tiene las siguientes características:

- El resultado principal del Análisis es el Documento de Especificación de Requisitos del Sistema. Si solamente es un sistema software, se llama Especificación de Requisitos Software.

- La ERS no debe ser ambigua.
- Debe ser completa y consistente.
- Debe ser fácil de verificar y fácil de modificar, y fácil de utilizar en la fase de explotación y mantenimiento.
- Debe ser fácil de identificar el origen y las consecuencias de los requisitos.

Dentro del análisis se engloba el modelado del sistema. Hay muchos tipos diferentes de modelos que se pueden elaborar, pero todos ellos son representaciones abstractas de lo que al final será una combinación de hardware y software que compone el sistema. La idea que subyace bajo el modelado es que los requisitos admitan una representación más técnica. Así, los modelos son representaciones gráficas de distintos aspectos del sistema.

Cada uno de los distintos modelos se usan en función de qué fase o etapa del proceso de ingeniería del software nos encontramos, análisis o diseño. Y mediante los mismos, se pueden especificar aspectos del sistema como los requisitos, la estructura y el comportamiento. Para realizarlos, se emplean *herramientas de modelado*, que ayudan a:

- Centrarse en las propiedades importantes del sistema, y restarle atención a las menos importantes.
- Discutir cambios y correcciones en los requisitos de usuarios, a bajo coste y riesgo mínimo
- Verificar que el analista comprenda correctamente el ambiente del usuario y que lo haya respaldado con información documental para que los diseñadores de sistemas y programadores puedan construir el sistema.

En el citado proceso de ASI de Métrica v3, se distingue entre análisis estructurado y orientado a objetos. A continuación, se exponen las técnicas de modelado más importantes, siguiendo la misma distinción que realiza Métrica v3 (para análisis estructurado y para orientado a objetos), ya que éstas cambian según se trate de uno u otro tipo de análisis.

Análisis estructurado

Dentro de este tipo existen dos tipos de modelado:

Modelado de las Funciones del Sistema: proporciona una visión funcional del dominio de información del problema. Establecen cómo los datos pasan a través del dominio de información. Se cuenta con las siguientes técnicas asociadas a este modelado:

- *Diagrama de Flujo de Datos (DFD):* Su objetivo principal es construir un modelo lógico del sistema que facilite su comprensión. El modelo final estará formado por entidades externas que aportan o reciben información del sistema, procesos que manipulan los datos, almacenes de datos como depósito de información y flujos de datos como mecanismo de comunicación entre los tres elementos anteriores.
- *Diccionario de Datos (DD):* Describen y referencian todos los objetos que aparecen en los DFDs, asegurando la consistencia de todos los objetos del sistema y facilitando la reutilización de dichos objetos.

Modelado de Datos Almacenados: proporciona una visión estática del dominio de información del problema. Identifican la estructura estática de las entidades de datos y las relaciones. Como técnicas asociadas a este modelado se cuenta con:

- *Modelo Entidad – Relación (E/R):* Representa el Modelo conceptual de datos. Es único para cada sistema de información y se corresponde con el nivel conceptual de la Arquitectura ANSI a tres niveles.
- *Diagrama de Estructura de Datos (DED):* Representa el Modelo lógico de datos. Depende del Sistema Gestor de Base de Datos que se vaya a utilizar en el sistema y por tanto no es único. Se corresponde con el nivel lógico global o externo de la Arquitectura ANSI.

Análisis orientado a objetos

En este tipo de análisis destacan como técnicas:

1. *La definición de Clases y la colaboración entre ellas.* En orientación a objetos, un programa consta de un conjunto de objetos, los cuales son abstracciones de cosas del mundo real. Cada objeto es un ejemplar de una clase, de modo que ésta última define a un conjunto de objetos que comparten una misma estructura y comportamiento común. Estas herramientas de modelado permiten realizar la definición de las mismas y las relaciones entre ellas.
2. *La definición de los Casos de Uso:* Un caso de uso es una descripción de la secuencia de interacciones que se producen entre un actor y el sistema, cuando el actor usa el sistema para llevar a cabo una tarea específica. Existen herramientas que ayudan a representar estas interacciones.
3. *Modelo de Dominio:* Se trata de un modelo conceptual que incluye todos los elementos de un problema específico en un dominio de interés.

2.2 Modelado de dominio

La esencia del análisis orientado a objetos es la descomposición del problema en conceptos individuales. Un modelo de dominio contiene principalmente los conceptos y relaciones significativos en el dominio del problema y no de la solución. Así, un modelo de dominio es una representación visual de clases conceptuales o de objetos reales en un dominio de interés y las relaciones presentes entre sí.

Las entidades (clases) contienen la lógica del negocio, y en ellas termina resolviéndose el sistema. Una diferencia esencial entre el análisis orientado a objetos y el estructurado es la división por clases conceptuales en lugar de la división por funciones. Por lo tanto, la principal tarea del análisis es identificar diferentes conceptos en el dominio del problema y documentar el resultado en diagramas de dominio.

El modelo de dominio se usa como base para el diseño de los objetos software y, de hecho, es uno de los artefactos más importantes del análisis. Podría ser considerado como un diccionario visual de abstracciones de clases conceptuales, vocabulario e información del dominio.

El modelo de dominio también es conocido como modelo conceptual, modelo de objetos del dominio o modelo de objetos de análisis. En UML se representa mediante un diagrama de estructura, el diagrama de clases, que es utilizado en dos fases del desarrollo de software:

- En el modelo conceptual o de dominio en la fase de análisis.
- En el diagrama de clases en la fase de diseño, para representar los elementos en el espacio de la solución software.

En UML se representa, por tanto, el modelo de dominio mediante el diagrama de clases, pero sin definición de operaciones ni asignación de responsabilidades de los objetos, que no forman parte de este modelo. En un modelo de dominio se pueden tener clases conceptuales sin atributos, o para las cuales no haya requerimientos de información a registrar, o bien modelar clases conceptuales con rol de comportamiento en lugar de información.

Para identificar las clases del modelo de dominio se puede utilizar una lista de categorías de clases conceptuales e identificar frases nominales. En la siguiente tabla se muestran algunos ejemplos:

<i>Categoría</i>	<i>Ejemplo</i>
Objetos tangibles o físicos	Casa, Avión
Especificaciones, diseños o descripciones de las cosas	PlanoDeLaCasa, EspecificaciónDelProducto, DescripciónDelVuelo
Lugares	Tiendo, Aula
Transacciones	Venta, Pago, Reserva, Tranferencia
Líneas de la transacción	LíneaDeVenta
Roles de la gente	Cajero, Piloto, Jefe
Contenedores de otras cosas	Aula, Ciolectivo, Lata, Mochila
Contenidos	Pasajero, Artículo, ÚtilEscolar
Otros sistemas informáticos o electromecánicos externos al sistema	SistemaDeAutorización, ControlDeTra´ficoAéreo
Conceptos abstractos	Amor, Celos, Ansia, Acrofobia
Organizaciones	DepartamentoDeVentas, CompañíaAérea
Hechos	Reunión, Vuelo, Aterrizaje, Venta, Pago
Procesos (normalmente no se representan como conceptos, pero podría ocurrir)	VentaDeUnProducto, ReservaDeUnAsiento (no confundir con transacciones)
Reglas y políticas	PolíticaDeReintegro, PolíticaDeCancelación
Catálogos	CatálogoDeProductos
Registros de finanzas, trabajo, contratos, cuestiones legales	Recibo, Remito, Factura, ContratoDeEmpleo, Expediente
Instrumentos y servicios financieros	LíneaDeCrédito, Stock
Manuales, documentos, artículos de referencia, libros	ManualDeReparaciones, ListaDeCambios
Relaciones	Amistad, Parentesco (podría estar en conceptos abstractos pero es bueno destacarlo ya que las relaciones no suelen ser consideradas al modelar)

Figura 1 Categorías para detección clases conceptuales

Se intenta identificar sustantivos o frases nominales en el vocabulario y descripciones del dominio del problema. No obstante, esta técnica no puede ser aplicada de manera sistemática, sino que es necesario aplicar el “sentido común” y capturar solamente las abstracciones que se consideren adecuadas, puesto que el lenguaje natural es habitualmente ambiguo y los conceptos relevantes no siempre se encuentran de manera explícita.

Una guía para construir un diagrama de dominio podría ser la siguiente:

- **Listar las clases conceptuales candidatas.** Los casos de uso pueden ser una fuente de referencia para identificar frases nominales, y modelo de dominio debe verse como un mapa de conceptos. No obstante, es necesario tener especial cuidado con no modelar clases que no están incluidas en la funcionalidad que se está describiendo, pues es fácil caer en el error de modelar todo lo conocido y no solamente el ámbito del dominio del problema.
- **Definir el nombre para cada clase.** Las clases se crean mediante elementos de tipo clase, a las cuales se les debe asignar un nombre descriptivo.
- **Añadir las asociaciones necesarias entre clases.** Una asociación es una relación entre clases que indica alguna conexión relevante. No obstante, en el modelo de dominio es más importante encontrar las clases que las relaciones entre las mismas. En un modelo de dominio con una determinada cantidad de clases puede ocurrir que haya muchas relaciones entre las mismas, por lo que quizás sea conveniente no modelar todas las relaciones posibles. Una buena práctica es repasar el diagrama de casos de uso, en caso de que ya se haya definido, y solamente mostrar las relaciones que se necesitan en el mismo. Es importante, particularmente en esta etapa, tener en cuenta que una clase no equivale necesariamente a una tabla del modelo de datos. Las asociaciones se representan con una línea entre clases y con un nombre de asociación. A diferencia de los diagramas de diseño, en el análisis, las asociaciones entre clases son bidireccionales, opcionalmente se puede utilizar una “flecha de dirección de lectura” que no tiene significado en términos del modelo, solamente es una ayuda para el lector del diagrama. Si la “flecha de dirección de lectura” no está presente, existe una convención de leer la asociación de izquierda a derecha o de arriba hacia abajo.

- **Definir los atributos necesarios para cada clase.** Una vez se han obtenido ya las clases con sus nombres, se les pueden incorporar los atributos que contendrán. Al igual que con las relaciones, resulta útil identificar aquellos atributos necesarios para satisfacer los requisitos del escenario que se está analizando.
- **Definir las clases de especialización.** En algunas situaciones, es una práctica muy útil identificar una clase que represente un concepto de modo general y luego identificar clases de ese concepto de modo especializado. Esta jerarquía es conocida con el nombre de generalización-especialización, o simplemente jerarquía de clases en la cual, la superclase representa un concepto más general y la subclase conceptos más especializados.

La notación y elementos que se utilizan en el modelo de dominio son explicados en detalle en el apartado del diagrama de clases del presente tema, ya que ambos utilizan el mismo tipo de diagrama.

Hay que tener en cuenta que no existe un único modelo de dominio correcto, pues todos serán una aproximación del dominio que estamos intentando entender. Un buen diagrama de dominio simplemente captura las abstracciones y proporciona la información necesaria para entender el contexto.

Un modelo de dominio suele contener varios diagramas. La regla general puede ser hacer un diagrama de dominio para cada caso de uso. Si bien aparecen situaciones donde un caso de uso no cuenta con demasiada gestión de clases y se vale de otros casos de uso para describir una funcionalidad, por lo que la tendencia es englobar en un único diagrama de dominio esa funcionalidad.

Por otro lado, la cantidad de clases puede llegar a ser lo suficientemente amplia para que sea conveniente dividirlos en paquetes que incluyan conceptos fuertemente relacionados. Esto ayuda para mejorar la comprensión y para abordar el trabajo de análisis en paralelo, donde cada analista genera el diagrama de dominio que acompañara al caso de uso. Al empaquetar las clases, el diagrama de dominio mostrará el intercambio que hay entre los diferentes paquetes. En epígrafes posteriores se verá en más detalle el diagrama de paquetes.

Como siempre, el objetivo de estos diagramas es servir como una herramienta para entender el sistema. Si el diagrama no resulta claro, no se estará cumpliendo con uno de los motivos de la modelización conceptual. Es por ello por lo que puede ser una buena práctica discutir los diagramas entre los analistas que estén haciendo la tarea y los mismos diseñadores que tomen estos diagramas como entrada para realizar el resto de modelos y especificaciones de diseño.

2.3 Modelo entidad relación

Un modelo de datos es una técnica que facilita la interpretación de un Universo de Discurso. Puede entenderse como un conjunto de conceptos, reglas y convenciones que permiten describir los datos de una parcela del mundo real. Al resultado de aplicar dicho “instrumento” sobre el mundo real se le denomina esquema o estructura de datos.

Los modelos de datos se pueden clasificar en dos tipos:

- Lógicos, que abarcan el nivel de usuario y el nivel de administrador.
 - Modelos conceptuales (modelo E/R), son independientes del Sistema de Gestión de Base de Datos (SGBD), con mayor capacidad semántica y mayor nivel de abstracción que los Modelos de Datos.
 - Modelos de Base de Datos, que son de distintos tipos, como jerárquico relacional.
- Físicos, que abarcan el nivel interno o nivel de máquina y que no están formalizados ni normalizados.

Modelar la información en un sistema aporta importantes ventajas a la hora de realizar el análisis. Las estructuras de datos y las relaciones pueden llegar a ser tan complejas que es preferible separarlas y examinarlas independientemente del proceso que se llevará a cabo.

El diagrama de entidad-relación es un modelo que describe con un alto nivel de abstracción la distribución de datos almacenados en un sistema. Es una herramienta muy efectiva de modelado, ya

que puede resultar muy útil a la hora de comunicarse con el grupo de administración de base de datos. Basándose en la información presentada por el diagrama, este grupo puede ver el tipo de claves, índices o apuntadores que se necesitarán para llegar de manera eficiente a los registros de las bases de datos.

De la misma manera, resulta muy apropiada para el analista, ya que enfatiza las relaciones entre almacenes de datos en el Diagrama de Flujo de Datos (DFD) o de otros modelos, que de otra forma se hubieran visto solamente en la especificación funcional.

Entre las distintas técnicas que hoy en día se emplean para la modelización semántica de datos a la hora de diseñar un Sistema de Información (SI) destaca por su extendida utilización el modelo Entidad – Relación (E/R), propuesto por Peter CHEN en 1976 en su ya histórico artículo para la ACM (Association of Computing Machinery): “*The Entity-Relationship Model - Toward a Unified View of Data*”.

Aunque en nuestro país es ampliamente conocido como modelo Entidad-Relación, algunos autores insisten en denominarlo modelo Entidad/Interrelación, con objeto de evitar confusión con la traducción en castellano del término “relation”, base del Modelo Relacional de Codd.

El modelo E/R es utilizado como técnica de representación gráfica de datos durante la fase de Análisis del Sistema de Información, como paso previo a su diseño y adecuación a un modelo físico y a un determinado Sistema Gestor de Base de Datos (SGBD). De hecho, es una de las técnicas incluidas en la metodología Métrica v3.

Permite diseñar el esquema conceptual de una base de datos y se basa en los conceptos de: entidad, tipo de entidad, atributo y relación, para finalmente plasmar toda esta información en los llamados diagramas entidad-relación. En los siguientes apartados se estudiarán estos conceptos.

2.3.1 Elementos del Modelo E/R

Los elementos fundamentales de que consta el Modelo E/R básico son los siguientes: entidades, asociaciones, relaciones o interrelaciones, atributos y restricciones o limitaciones de integridad. A continuación, se detallan cada uno de ellos.

2.3.1.1 Entidades

Una entidad es todo aquello de lo que se desea almacenar información. Caracteriza a un tipo de objeto del problema a modelizar. En sentido estricto, se denomina “tipo de entidad” a la estructura genérica y “entidad” a cada una de las *instancias* de ese tipo de entidad. Una instancia es cada uno de los elementos con características comunes que componen una entidad. No obstante, en adelante se utilizará el término entidad para hacer referencia a “tipo de entidad”, pues generalmente se hace ese uso.

Según Métrica v3, se define entidad como cualquier objeto real o abstracto, que tiene existencia por sí mismo, se puede identificar de una manera clara y precisa, y sobre el que se quiere tener información en el sistema. Por ejemplo, si nuestro universo del discurso es una empresa, “Empleado” sería una entidad, puesto que parece lógico que se quiera tener información al respecto en el sistema.

Toda entidad debe ser distinguible del resto de las entidades, cada una de ellas debe poder identificarse de manera única, y tiene nombre y posee atributos definidos en un dominio determinado.

Se pueden considerar dos clases de entidades:

- *Regular o Fuerte*, aquellas que tienen existencia propia y atributos clave propios.
- *Débil*, cuya existencia depende de la de una entidad regular y carecen de atributos clave propios.

La representación gráfica de una entidad consiste en un rectángulo etiquetado con el nombre del tipo de entidad. Para distinguir las entidades débiles, se representa su tipo de entidad mediante dos rectángulos concéntricos. Las entidades se nombran con un sustantivo.



Figura 2 Representación de entidad regular y entidad débil

2.3.2 Relaciones

Una relación es una asociación o relación matemática entre varias entidades, que representa un conjunto de conexiones entre objetos. Cada instancia de la relación representa una asociación entre cero o más ocurrencias de un objeto y cero o más ocurrencias del otro.

Al igual que en el caso de las entidades, se tienen asociaciones o relaciones *regulares* o *fuertes* y *débiles*. Las primeras son aquellas que relacionan entidades fuertes, mientras que las segundas relacionan una entidad débil con una fuerte.

Los tipos de interrelación se representan mediante un rombo etiquetado con el nombre de la interrelación, unido mediante arcos a los tipos de entidad que asocian. Normalmente se nombran con un verbo. Para distinguir las asociaciones débiles, se representa mediante dos rombos concéntricos.



Figura 3 Representación de una relación

El *grado de la relación* o de la *interrelación* representa el número de entidades que participan en una asociación. Teniendo en cuenta esta propiedad, se tiene que las relaciones pueden ser *unitarias* (entre una misma entidad), *binarias* (entre dos entidades) o *N-arias* (entre N entidades).

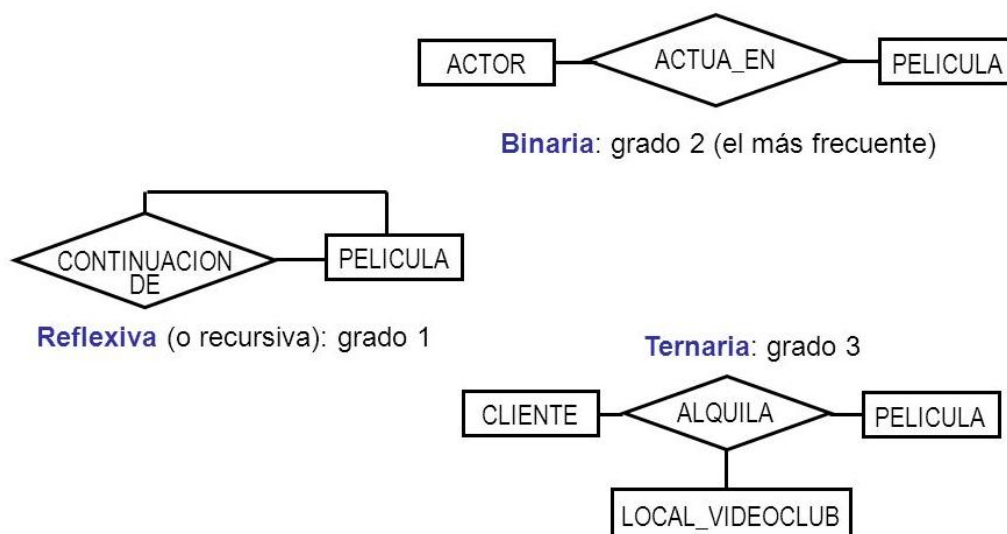


Figura 4 Representación del grado de una relación

2.3.3 Atributos.

A cada una de las posibles propiedades o características que tiene un tipo de entidad o un tipo de interrelación se le denomina “atributo”. Cada atributo toma valores de uno o varios dominios, de modo que un “dominio” es el conjunto de posibles valores que puede tomar una cierta característica.

Los atributos se representan gráficamente etiquetando con su nombre un círculo u óvalo unido al tipo de entidad o interrelación mediante un arco. Alternativamente, se pueden representar mediante un pequeño círculo y el nombre a la derecha del mismo. El identificador aparece con el nombre marcado o subrayado, o bien con su círculo en negro. En general, se representan solamente los atributos más significativos.

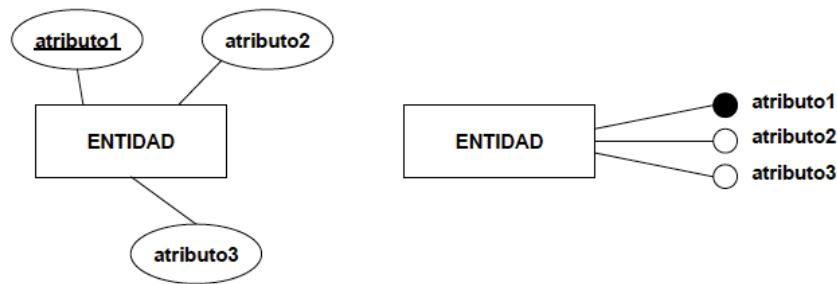


Figura 5 Representaciones de los atributos.

Conceptualmente, cada entidad y cada relación son distintas, y la diferencia entre distintas entidades y distintas relaciones debe expresarse en términos de sus atributos. De modo que toda entidad o relación debe ser unívocamente identificada y distinguible mediante un atributo, o conjunto de atributos, denominado **identificador, clave principal o primaria**. Para entender este concepto hay que tener en cuenta una serie de definiciones previas:

- Se denomina *Superclave* a cualquier conjunto de atributos que permita distinguir a todas las entidades de cualquier instancia válida de un tipo de entidad.
- Se denomina *Clave candidata* de un tipo de entidad a una superclave que no contiene ningún subconjunto que también sea superclave (Conjunto mínimo de atributos que forma una superclave).
- Finalmente, se tiene que la *Clave primaria* es una clave candidata seleccionada por el diseñador para distinguir entre las entidades de cada instancia. Dicho de otra forma, la clave principal es aquel atributo o conjunto de atributos cuyos valores identifican unívocamente cada entidad. La representación gráfica del atributo principal o clave es mediante una línea que lo subraye.

Tipos de atributos

- **Atributos monovalorados y multivalorados.** Se llaman atributos multivalorados a aquellos que pueden contener más de un valor simultáneamente, y monovalorados a los que solamente pueden contener un valor. A modo de ejemplo, una persona puede tener varios números de teléfono (casa, trabajo, móvil), donde se contaría con un atributo multivalorado “teléfono”.
- **Atributos simples y compuestos.** Se dice que un atributo es compuesto cuando puede descomponerse en otros componentes o atributos más pequeños, y simple en otro caso. En el caso del domicilio puede que interese descomponerlo a su vez en calle, el número y la ciudad por separado.



Figura 6 Representación de atributos

2.3.4 Restricciones

Con los elementos anteriores se han definido los elementos principales del modelo E/R. Sin embargo, en el modelo E/R también se pueden definir numerosas *restricciones* sobre los tipos de entidades y tipos de relaciones. Las *restricciones* son propiedades que se asocian a un tipo de entidad o de relación. Las *instancias válidas* del tipo de entidad o relación son aquellas en las que se cumplan el conjunto de restricciones asociadas.

A modo de ejemplo, para entender qué se entiende por restricción, se plantea la siguiente situación. Supongamos una entidad profesor y una entidad alumno, que se encuentran relacionadas por la asociación supervisa. Si se quiere que un profesor supervise de manera única a un alumno habrá de indicarlo en el diagrama de manera que no puedan existir dos profesores que supervisen el mismo alumno. Esto sería una restricción en el modelo.



Figura 7 Ejemplo de restricción en modelo E/R

Se tienen las siguientes restricciones.

- **Cardinalidad:** Es el número de instancias o elementos de una entidad que pueden asociarse a un elemento de la otra entidad relacionada. Se representa mediante una pareja de datos, en minúsculas y entre paréntesis, asociada a cada uno de las entidades que intervienen en la relación. Se tienen: (0,1), (1,1), (0,n), (1,n), (m,n), siendo el primer valor el número de instancias mínimo, y el segundo valor el número máximo.
 - Una N o M significa que puede tener varias instancias.
 - Un 0 significa que puede estar vacío, es decir, no tener ninguna instancia y por lo tanto no tiene un número mínimo obligatorio.
 - Un 1 significa que solamente tiene una instancia.
- **Cardinalidad máxima de la relación o Tipo de correspondencia.** Se toman los máximos de las cardinalidades que intervienen en la relación. Es el número de máximo de instancias de cada entidad que pueden participar en una relación. Hay 3 tipos posibles:
 - **1:1.** Una instancia de una entidad se relaciona con una instancia de la otra. Por ejemplo, que un país pertenece a un continente o el mostrado en la figura 9, que un profesor solamente pueda supervisar a un alumno.
 - **1:N.** Una instancia de una entidad se relaciona con varias instancias de la otra. Por ejemplo, que un cliente pueda realizar varios pedidos.



Figura 8 Ejemplo de cardinalidad 1:N

- *N:N*. Varias instancias de una entidad se relacionan con varias de la otra. Por ejemplo, que una persona posea varias viviendas y que las viviendas puedan tener varios propietarios.

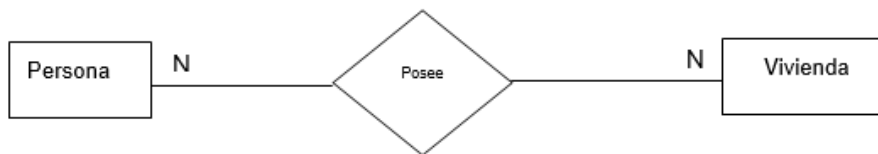


Figura 9 Ejemplo de cardinalidad N:N

- *Restricciones semánticas*. Una restricción semántica es una proposición que puede tomar el valor de “cierto” o “falso” para cada ocurrencia de un tipo de entidad, de un tipo de interrelación o de un conjunto de estos elementos. Se clasifican en:
 - Restricciones estructurales, inherentes al modelo entidad relación, como son por ejemplo los tipos de relaciones, de entidades y de atributos.
 - Restricciones de usuario, propias del Universo de Discurso que se desea modelizar. A su vez, estas últimas pueden definirse sobre valores, delimitando los valores posibles para un cierto objeto del modelo (entidad, dominio, atributo), o bien sobre interrelaciones, delimitando el número de objetos (entidades, tipos de entidades) que pueden intervenir dentro de un tipo de interrelación. La definición de restricciones de usuario permite aportar más semántica al esquema conceptual que se construye, así como asegurar la integridad de los datos que se recogen, informando sobre estados y transiciones prohibidos en nuestro Universo de Discurso. Ejemplos de restricciones de usuario son, por ejemplo, que todo profesor deba pertenecer a un departamento concreto o que las clases de una asignatura se imparten solamente en días, horas y aulas determinadas.

A la hora de diseñar el modelo E/R deben tenerse en cuenta algunas cuestiones de diseño, como son:

- No repetir nombres en los elementos.
- Definir atributos claves para todas las entidades.
- Los atributos que se definen para cada entidad son propios de esa entidad y la relación que se establece entre claves de entidades permite acceder a esos atributos.

2.3.5 Modelo Entidad-Relación extendido.

Los conceptos básicos del modelo E-R pueden modelar la mayoría de las situaciones, pero algunos aspectos se pueden mostrar más adecuadamente con el modelo E-R extendido, que presenta nuevas características para dotar al modelo de más herramientas para reflejar las necesidades del sistema, algunas de las cuales se exponen a continuación.

2.3.5.1 Especialización - Generalización

Un tipo de entidad E es una generalización de un tipo de entidad R cuando los atributos de E están incluidos en los atributos de R. La entidad E recibe el nombre de **Superentidad**, mientras que R es conocida como **Subentidad**. En la siguiente imagen se muestra un ejemplo de generalización de Cuenta, que puede ser una Cuenta de Ahorro o una Cuenta Cheque.

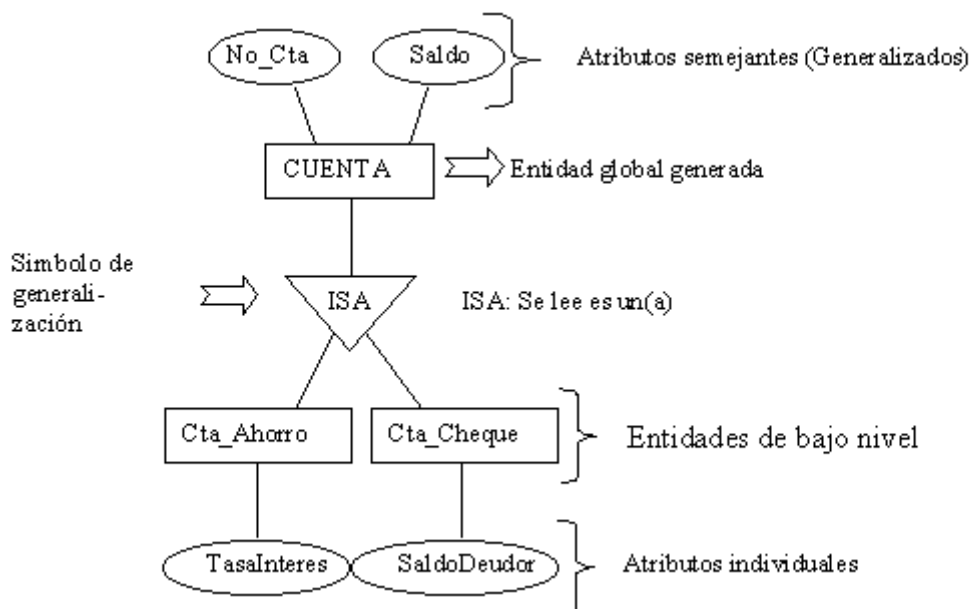


Figura 10 Generalización en modelo ER

La generalización se representa con un triángulo que incluye el texto “is a” o “es un”, entre la superentidad y sus correspondientes subentidades, unidos mediante arcos.

La representación de una jerarquía entre tipos de entidad (superentidades - subentidades) puede presentarse de varias maneras:

- **Jerarquías (en general):** Cada subentidad “hereda” todas las relaciones y atributos de la superentidad
- **Jerarquías según cobertura:**
 - **Jerarquía con Cobertura Total.** Cada elemento de la superentidad debe estar en al menos una de las subentidades.
 - **Jerarquía con Cobertura Parcial.** Puede haber elementos de la superentidad que no estén en ninguna de las subentidades.
- **Jerarquías según solapamiento:**
 - **Jerarquía Disjunta.** Cada elemento de la superentidad está a lo sumo en una subentidad.
 - **Jerarquía con Solapamiento (No disjunta).** Los elementos de la superentidad pueden estar en más de una subentidad.

La representación de las jerarquías se realiza mediante un triángulo invertido, con la base paralela al rectángulo que representa el supertipo y conectando a éste y a los subtipos. Si la división en subtipos viene determinada en función de los valores de un atributo discriminante, éste se representará asociado al triángulo que representa la relación. En el triángulo se representará: con una letra **d** el hecho de que los subtipos sean disjuntos, con un círculo o una **O** si los subtipos pueden solaparse y con una **U** el caso de uniones por categorías. La presencia de una jerarquía total se representa con

una doble línea entre el supertipo y el triángulo. En la siguiente imagen se muestra su representación gráfica.

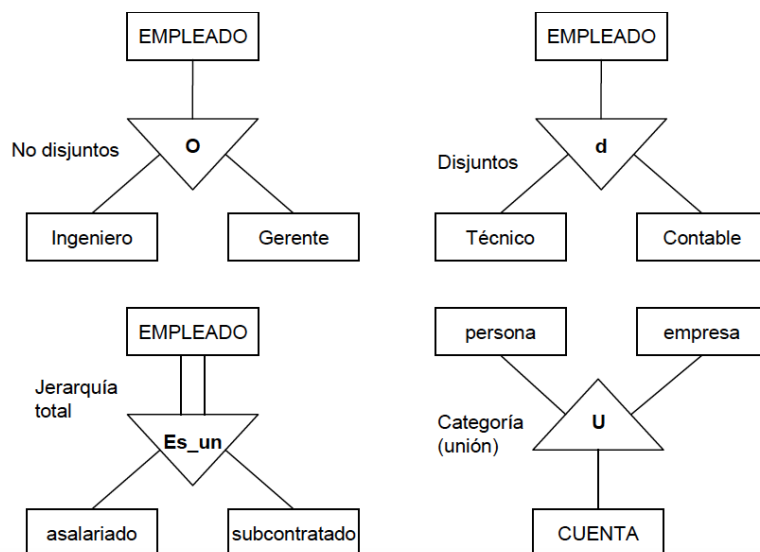


Figura 11 Representación gráfica de jerarquías en E/R

2.3.5.2 Cardinalidad mínima de un tipo de entidad

En el nuevo modelo extendido se tiene en cuenta la *cardinalidad mínima* de la entidad, detallada anteriormente. Ésta se define como el número mínimo de ocurrencias de un tipo de entidad que se pueden interrelacionar con cada ocurrencia de los otros tipos de entidad en un determinado tipo de interrelación.

2.3.5.3 Dependencia en existencia y en identificación

En relación a las entidades débiles, se cuenta con dos conceptos nuevos en el modelo extendido: el de dependencia en existencia y dependencia en identificación. Se dice que un tipo de entidad débil *depende en existencia* de otro tipo de entidad cuando las ocurrencias de aquél no tienen sentido sin la ocurrencia de éste. Si además de lo anterior, las entidades débiles no se pueden identificar únicamente mediante los valores que reciben sus propios atributos, sino que necesitan algún atributo de las entidades con las que se relaciona, entonces se trata de una *dependencia en identificación*.

2.3.5.4 Agregación

La agregación se trata de considerar un conjunto de elementos, ya sean entidades o relaciones, como si fueran un único tipo de entidades. Surge cuando varias entidades, que no han de ser obligatoriamente distintas, se asocian de forma que se pueden ver como una única, la cual se puede relacionar con otras entidades. Se denota incluyendo en un rectángulo todos los componentes de la agregación, pues el modelo E-R no permite establecer relaciones entre relaciones.

Por ejemplo, si se quieren gestionar las incidencias de una clase. Se cuenta con una entidad profesor que imparte clases a un grupo en un aula. En una clase habrá ninguna o varias incidencias, e interesa determinar qué participantes ha habido en esa clase con incidencia, qué profesor, en qué aula y con qué grupo. Atendiendo a la relación de agregación se podría crear una nueva entidad clase que englobe todo, representándose de la siguiente manera:

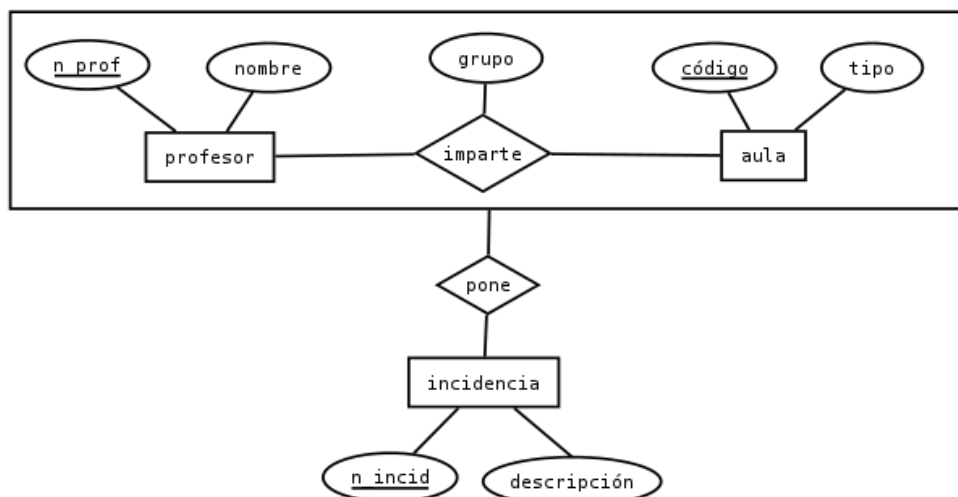


Figura 12 Representación de una agregación

2.3.6 Traducción del modelo E/R al modelo relacional

Un modelo conceptual puede ser transformado a varios modelos lógicos, entre ellos el modelo relacional. Para obtenerlo, se ha de partir de un modelo E/R o modelo conceptual previamente tratado. No se emplea el modelo E/R en el Sistema de Gestión de Base de Datos directamente, sino que previamente ha de generarse a partir de éste un modelo lógico equivalente, es decir, que cumpla con la estructura y restricciones del modelo relacional. Así, el objetivo principal de esta traducción o transformación es el paso de un diagrama del modelo E/R a tablas del modelo relacional, casi implementables en lenguaje SQL. En primer lugar, se realiza una reestructuración del modelo, eliminando redundancias y generalidades, para luego transformar el modelo resultante en el modelo relacional.

Para llevar a cabo la reestructuración del modelo E/R hay que realizar un *Análisis de Redundancias*, donde se analizan aquellos atributos cuyos valores pueden ser derivados de otros de la misma entidad, o atributos que son fácilmente derivables de otras entidades mediante funciones de agregación, para tratar de eliminar las redundancias. La redundancia presenta como principal inconveniente la necesidad de mayor capacidad de almacenamiento y más operaciones para mantener la información actualizada.

El segundo paso de la reestructuración es la eliminación de *generalidades*, ya que el modelo relacional no puede representarlas directamente. Existen distintas alternativas para evitarlas: plegar las entidades hijo hacia el padre, o sustituir la generalización por más relaciones.

Una vez se cuente con este modelo conceptual equivalente, el paso al modelo relacional equivale a realizar los siguientes pasos:

- *Transformaciones de entidades fuertes.* Construir tablas por cada una de las entidades que aparecen en el modelo E/R, que tengan el mismo nombre, con los mismos atributos y clave. Los atributos pasan a ser columnas de la tabla.
- *Transformaciones de las relaciones.* En caso de que sea necesario, crear tablas para las distintas asociaciones, con el mismo nombre, con los atributos de la relación, y las claves de las entidades vinculadas, que formarían la clave de la tabla, aunque hay que distinguir según el tipo de relación. En los casos en los que no es necesario crear una tabla adicional para la relación, se crearán las claves ajenas en las tablas correspondientes:

- *Relaciones 1:N*, se propaga el identificador de la entidad de cardinalidad máxima 1 a la que es N, teniendo en cuenta que:
 - Si la relación es de asociación, la clave propagada es clave ajena en la tabla a la que se ha propagado.
 - Si la relación es de dependencia, la clave primaria de la tabla correspondiente a la entidad débil está formada por la concatenación de los identificadores de ambas entidades.
 - *Relaciones 1:1*, es un caso particular de las 1:N y, por tanto, se propaga la clave en las dos direcciones. Se debe analizar la situación, intentando recoger la mayor semántica posible, y evitar valores nulos.
 - *Relaciones N:M*, se transforman siempre en una tabla. se transforman en una tabla, cuya clave primaria es la concatenación de los identificadores de las clases asociadas, siendo cada uno de ellos clave ajena de la propia tabla. Si la relación tiene atributos, éstos se transforman en columnas de la tabla.
 - Las relaciones de agregación se transforman del mismo modo que las 1:N.
- *Transformaciones de las entidades débiles*. Toda entidad débil incorpora una relación con una entidad fuerte. Esta relación no necesita incorporarse como tabla en el modelo relacional. Sí se necesita incorporar la clave de la entidad fuerte como clave en la entidad débil. Es más, normalmente esa clave externa forma parte de la clave principal de la tabla que representa a la entidad débil.

2.3.7 Ventajas y desventajas del Modelo Entidad-Relación

Una de las ventajas del modelo E/R es que es una aproximación de alto nivel que permite expresar con mucha precisión el esquema conceptual. A su vez, los diagramas de E/R permiten mantener una visión global del sistema, lo que favorece la comunicación entre los analistas y diseñadores.

No obstante, este modelo también presenta algunos inconvenientes. Se puede citar como desventaja el hecho de que carece de un soporte formal. Además, los Sistemas Gestores de Bases de Datos (SGBD) no suelen implementarlo directamente, sino que previamente hay que realizar una transformación de este modelo E/R a un modelo de más bajo nivel que es el que corresponde a los SGBD, y que éstos comprenden. Generalmente, tal y como se ha visto, se suele transformar en un modelo relacional, que suele ser el más empleado como modelo de bajo nivel.

2.4 Modelos de clases

En el paradigma de la orientación a objetos, el modelo de clases se puede considerar la piedra angular sobre la que giran el resto de elementos, pues un sistema software bajo este paradigma está formado por un conjunto de objetos que cooperan para lograr el resultado final. Para un desarrollador, la programación orientada a objetos significa que debe enfocar su trabajo hacia los datos que trata su sistema y los métodos para manipular esos datos, en vez de pensar en los procesos de modificación de los flujos de datos que existen en el sistema.

Este planteamiento es opuesto a una visión algorítmica e imperativa (descomposición funcional) del sistema a implementar, es decir, hay un cambio de mentalidad en los programadores y en los ingenieros del software sobre cómo diseñar y construir un sistema software.

En la siguiente tabla se muestra un ejemplo de descomposición funcional comparada con el mismo ejemplo en orientación a objetos.

Descomposición funcional	Orientación a Objetos
<pre> programa () { dibujar_circulo(); ... Dibujar_linea(); } Dibujar_linea() { ... } </pre>	<pre> Class BarraHerramientas { Main () { Circulo circuloA = New Circulo(); circuloA.dibujar(); ... Linea LineaA = New Linea(); lineaA.dibujar(); Cuadrado CuadradoA = New Cuadrado(); CuadradoA .dibujar() ... } } </pre>
• Módulos contruidos alrededor de las operaciones. • Datos globales o distribuidos entre módulos. • Entrada/Proceso/Salida. • Diagramas de Flujo de Datos.	• Módulos contruidos alrededor de las clases. • Clases débilmente acopladas y sin datos globales. • Encapsulación/ Mensajes. • Diagramas jerárquicos de clases.

En este sentido, el diagrama de clases es un diagrama de estructura de UML que pretende mostrar la estructura interna del sistema diseñado al nivel de clases e interfaces, con sus características, restricciones y relaciones, asociaciones, generalizaciones...etc.

Los conceptos principales del modelo de clases son las clases y sus métodos, los objetos, las relaciones y los mensajes que intercambian entre ellos.

2.4.1 Clases

Una clase representa el conjunto de objetos que tienen unas características comunes. Los procedimientos y datos que resumen dichas características comunes de todos los objetos reciben el nombre de *métodos* y *atributos*, respectivamente. Los procedimientos operan sobre los datos de la clase, es decir, los métodos operan sobre los atributos de la clase.

Desde el punto de vista de la programación, se puede considerar a una clase como un “*tipo de datos de los disponibles*” en el sistema, como pueden ser “*integer*”, “*float*”, “*string*”, etc. Es decir, se podrán definir “variables” de ese tipo, que serán “objetos” de la clase indicada. Cada uno de los objetos individuales pertenecientes a una clase se denomina *instancia*.

Así, podría entenderse una clase como una plantilla con las características y comportamientos comunes de los objetos lo que permite definirlos y crearlos.

Atributos de la clase

Las clases son un concepto estático definido en el programa fuente, mientras que los objetos (instancias de una clase) son entes dinámicos que ocupan memoria en la ejecución de un programa. Esta cualidad permite definir *variables de clase* frente a las *variables de instancia* o atributos de clase y atributos de instancia (o de objeto). Estos términos son, por tanto, equiparables.

En el siguiente fragmento de código se indican ejemplos de variables de clase y de instancia.

```

public class SerHumanoBis{
    String nombre;
    String colorOjos;
    int edad;

    /*
     * Declaración e inicialización de una variable de instancia estática
     * Tiene sentido declararla estática pues todos los objetos
     * de la clase, teniendo en cuenta que ésta modela a un ser humano,
     * habitan en el mismo planeta
     */
    static String planeta="Tierra";

    void mostrarCaracteristicas(){
        System.out.println(nombre+" tiene "+edad+" a"+(char)164+"os");
        System.out.println("Sus ojos son "+colorOjos);
        System.out.println("Su planeta es la "+planeta);
    }

    void eresMayorEdad(){
        if(edad>=18){
            System.out.println(nombre+" es mayor de edad");
            System.out.println("Tiene "+edad+" a"+(char)164+"os");
        }
        else{
            System.out.println(nombre+" es menor de edad");
            System.out.println("Tiene "+edad+" a"+(char)164+"os");
        }
    }

    public static void main(String args[]){

```

Variable de Instancia

Variable de Clase / Estática

Figura 13 Ejemplo de código Java con declaración de variables

Una *variable de clase* o miembro de dato estático es una variable, al contrario que las variables de instancia, propia de la clase que la contiene, y no de instancias de la misma, esto quiere decir que todos los objetos que se creen de esta clase, comparten su valor, por lo que son llamadas variables estáticas. De hecho, en muchos lenguajes de programación se utiliza la palabra reservada "*static*" para indicar este hecho.

Por ejemplo, en Java, para invocar una variable de clase no es necesario crear un objeto de la clase pero si se quiere llamar desde otra clase es necesario escribir el nombre de la clase que la contiene seguido de un punto y el nombre de la variable.

Los atributos o variables de clase existen independientemente de que se haya creado o no el objeto de una determinada clase.

Por otra parte, como se verá en el apartado de visibilidad, la asignación directa de valores a atributos no es una buena práctica, y se debería realizar a través del método correspondiente.

Una *variable o atributo de instancia* es una variable que se relaciona con una única instancia de una clase. Cada vez que se crea un objeto, el sistema crea una copia de todas las variables que están vinculadas con dicha clase, haciéndolas propias de esa instancia. Solamente se puede acceder a ellas a través del identificador del objeto. Desaparecen cuando el objeto desaparece y se libera la memoria del sistema.

Estas variables son declaradas fuera del cuerpo de los métodos y dentro del de la clase, son de tipo global. Por lo tanto, pueden ser utilizadas por cualquier método no estático de dicha clase.

A este conjunto de variables, de instancia y de clase, se les denomina *atributos* y son los elementos que definen el estado de los objetos.

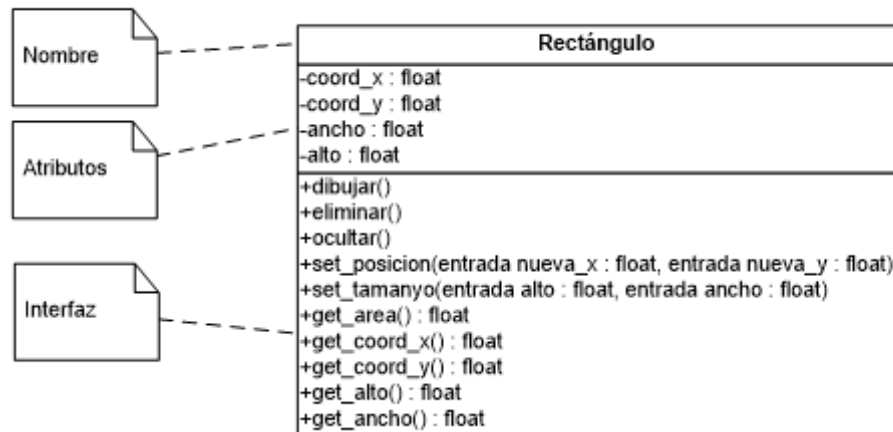


Figura 14 .- Clase Rectángulo con información de ámbito en los atributos y métodos

Métodos

Los métodos definen la forma en que los datos contenidos en los objetos son manipulados. El interfaz de la clase de objetos estará definido por el conjunto de métodos que soporta y los mensajes que es capaz de tratar (su protocolo).

Los tipos de operaciones más habituales son:

- ✓ Constructor: crea una instancia de la clase; puede haber varios constructores distintos para una misma clase, de copia, con atributos, por defecto o sin atributos, etc.. Tiene el mismo nombre de la clase. Por ejemplo si la clase se llama figura, ejemplos de constructores son: figura (figura a); figura (), figura (punto Centro, Area area_valor, Tipo_figura Tipo). El nombre es el mismo, varían los atributos de los constructores (en orden, tipo, y/o número)
- ✓ Destructor: destruye una instancia, elimina toda la referencia al objeto en memoria. La sintaxis puede variar de un lenguaje a otro, pero se suele representar como "~Nombre_Class();". Solamente hay un destructor.
- ✓ Selector (get): selecciona una parte del estado o devuelve una propiedad resultante de la combinación de algunos atributos; get_coord_x(), get_coord_y(), get_area(), ...
- ✓ Modificador (set): modifica una parte del estado según algún criterio; set_area(Double area);
- ✓ Copiador (copy, clone): copia un objeto en otro.
- ✓ Iterador: permite recorrer una estructura de datos (ej.: para recorrer una lista).
- ✓ Visualizador (view): muestra todo el estado del objeto de una forma elaborada.

Además, una clase tendrá aquellos métodos que proporcionen funcionalidades de la propia clase, relativos a su negocio o necesidades específicas.

Los métodos de un objeto se invocan exclusivamente con el mensaje adecuado, y al ser invocado un método de un objeto, solamente se referirá a la estructura de datos de ese objeto y no a la de otros, aunque sean de la misma clase.

Al invocar al método se hará de la forma reflejada en la figura 13, donde el objeto shb1 de la clase *SerHumanoBis* posee el método "*mostrarCaracteristicas()*". Es decir, hay una relación directa entre métodos y mensajes, ya que, para poder recibir un mensaje tiene que poseer dicho método.

Visibilidad de los atributos o propiedades de una clase

Los atributos y operaciones pueden tener los siguientes tipos de acceso:

- + Público: Se pueden acceder desde cualquier clase.

– Privado: Solamente se pueden acceder desde operaciones de la clase.

Protegido: Solamente se pueden acceder desde operaciones de la clase o de clases derivadas.

Los símbolos +, - y # representan gráficamente el ámbito público, privado y protegido respectivamente. Las herramientas CASE a veces utilizan otro tipo de representación gráfica del ámbito o tipos de acceso.

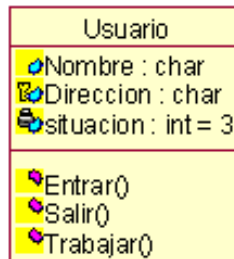


Figura 15 .- Clase con ámbito diseñada con una herramienta CASE

En general, se recomienda que los atributos se definan como privados y los métodos que constituyan el interfaz como públicos, estableciendo el resto como privados. No obstante, siempre se debe adecuar la visibilidad según las necesidades específicas de cada proyecto.

Además de los tipos de acceso señalados, existe el concepto de “*amistad*” entre clases, el cual permite definir un comportamiento intermedio entre público y privado complementario al de ámbito “*protegido*”, por ejemplo, cuando se referencian clases que están dentro del mismo paquete.

Para que una función externa a la clase pueda acceder a los atributos privados, algunos lenguajes como “C++” permiten definir dicha función como amiga de la clase.

Mensajes

Un mensaje es una solicitud que se le pide a un objeto para que se comporte de una manera determinada. Contiene el nombre de objeto al que va dirigido, el nombre de la operación y sus parámetros.

Al conjunto de mensajes al que puede responder un objeto se le llama *protocolo de objeto*.

En la imagen siguiente se puede ver como al objeto “shb1” se le invoca para que “Muestre sus características”, con el mensaje “*mostrarCaracteristicas ()*”.

```
public static void main(String args[]){
    SerHumanoBis shb1=new SerHumanoBis();
    shb1.nombre="Jesus";
    shb1.colorOjos="azules";
    shb1.edad=28;
    shb1.mostrarCaracteristicas();
    shb1.eresMayorEdad();

    System.out.println("-----");

    SerHumanoBis shb2=new SerHumanoBis();
    shb2.nombre="Rebeca";
    shb2.colorOjos="verdes";
    shb2.edad=27;
    shb2.mostrarCaracteristicas();
    shb2.eresMayorEdad();
}
```

Mensaje “mostrarCaracteristicas ()
al objeto “shb1”

Figura 16 .- Mensaje enviado a un objeto para mostrar sus características.

Objetos

Un objeto es una instancia particular de una determinada clase (un ejemplar de la clase) que tiene vida propia y que mantiene un estado (los datos o atributos del objeto se dice que mantienen su estado) durante el tiempo de vida que persiste el objeto.

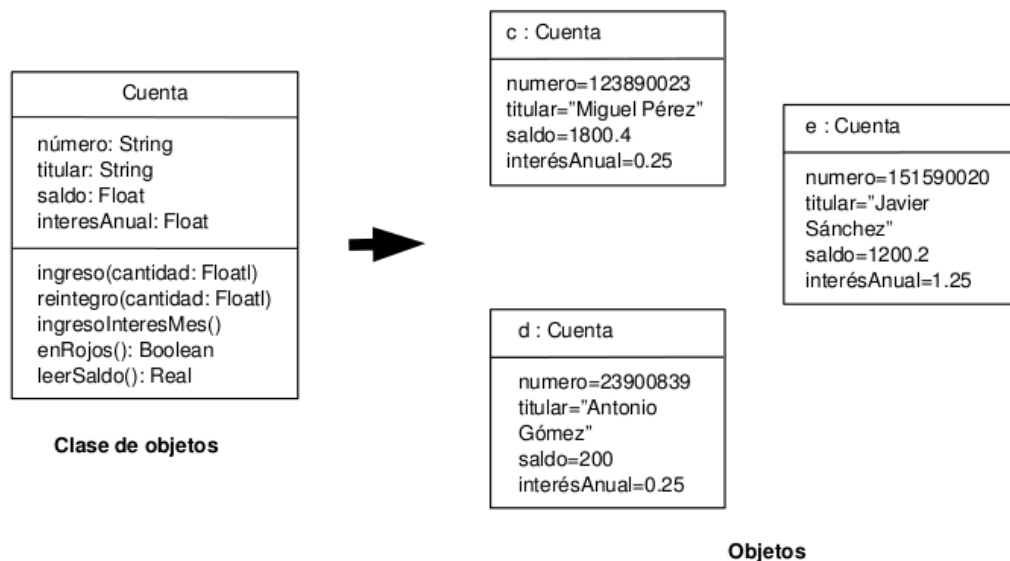


Figura 17 .- Relación entre una clase “cuenta” y los objetos o instancias de la misma

Ambos, los datos y los métodos, describen el estado y el comportamiento de un objeto respectivamente. Por lo tanto, los objetos poseen una funcionalidad (las operaciones que son capaces de hacer o los mensajes a los que son capaces de reaccionar) y un estado definido por sus atributos (datos con sus valores).

En un sistema Orientado a Objetos se representarán todos aquellos objetos que sean relevantes para el universo de discurso sobre el que trate el sistema.

Clases abstractas

Una clase que no tenga instancias se denomina clase abstracta.

- ✓ Una clase abstracta sirve para declarar las propiedades de un conjunto determinado de clases pero que a ese nivel de abstracción no se pueden concretar.
- ✓ Una clase es abstracta cuando no define alguna parte de la abstracción de datos que encapsula, p.e., un procedimiento virtual (declarado, pero no definido). Estos serán implementados por las clases hijas o subclases que sí serán clases concretas.

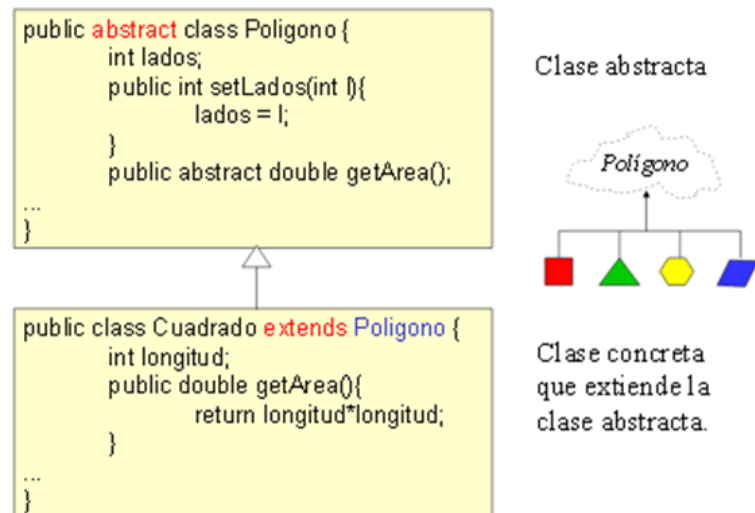


Figura 18.- Clase Abstracta Poligono y subclase Cuadrado que hereda de Poligono

Cada objeto puede heredar la estructura de datos y operaciones definidas para los objetos de una clase más amplia a la que pertenece, constituyendo una subclase dentro de ésta (la superclase). Una de las utilidades de las clases abstractas es la capacidad de heredar propiedades.

Esta característica de las clases, se representa en el modelo de clases mediante la palabra <<abstract>> en el nombre de la clase.

2.4.1.1 Relaciones entre clases

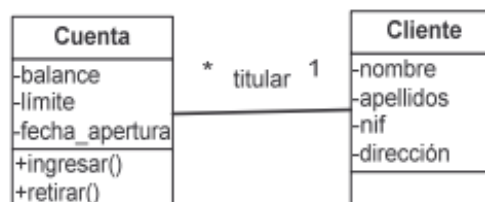
Las relaciones existentes entre las distintas clases indican cómo se comunican los objetos de esas clases entre sí. Es decir, los mensajes “navegan” por las relaciones existentes entre las distintas clases.

Los tipos posibles de relaciones entre clases son:

- **Asociación:** Es una relación estructural que describe una conexión entre objetos. Gráficamente se muestra como una línea continua que une las clases relacionadas entre sí. Aunque las asociaciones suelen ser bidireccionales (se pueden recorrer en ambos sentidos), en ocasiones es deseable hacerlas unidireccionales (restringir su navegación en un único sentido).



Asociación unidireccional



Asociación bidireccional

Figura 19 Relaciones de asociación en diagrama de clases

En una relación de asociación, ambos extremos de la línea pueden conectar con la misma clase, indicando que una instancia de una clase, está asociada a otras instancias de la misma clase, lo que se conoce como *asociación reflexiva*.

Las asociaciones tienen una multiplicidad, que determina cuántos objetos de cada tipo intervienen en la relación. Así, para especificarla, hay que indicar la multiplicidad mínima y máxima, tal y como se muestra en la siguiente tabla:

Multiplicidad	Significado
1	Uno y sólo uno
0..1	Cero o uno
N..M	Desde N hasta M
*	Cero o varios
0..*	Cero o varios
1..*	Uno o varios (al menos uno)

Figura 20 Multiplicidad de una clase

Dos casos particulares de relación son la *Agregación* y la *Composición*. En la agregación se representa que las partes pueden formar partes de distintos agregados, mientras que en la composición se representa una agregación disjunta y estricta, es decir, donde las partes solamente existen asociadas al compuesto. En las siguientes imágenes se muestran sus representaciones gráficas.

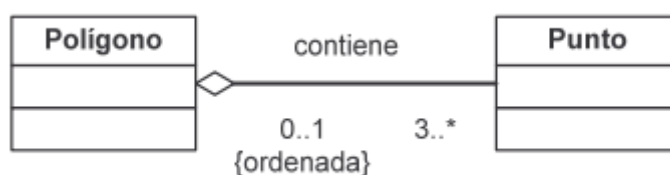


Figura 21 Relación de agregación en diagrama de clases

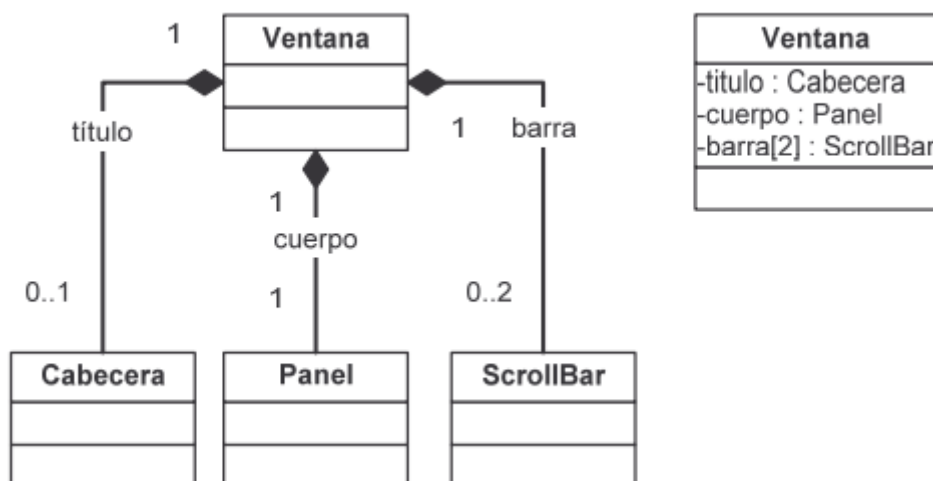


Figura 22 Relación de composición en diagrama de clases

- *Dependencia*: Se trata de una relación, más débil que la de asociación, que muestra la relación entre un cliente y el proveedor de un servicio usado por el cliente, siendo cliente el

objeto que solicita el servicio y servidor el objeto que provee el servicio solicitado. Gráficamente se representa con una línea discontinua con una punta de flecha que apunta del cliente al proveedor.



Figura 23 Relación de dependencia en modelo de clases

- **Generalización/especialización (herencia):** Las clases se pueden organizar de forma jerárquica, de manera que una subclase puede heredar las propiedades (datos y operaciones) de su superclase o modificarlas si fuese necesario. Una jerarquía de clases siempre tiene una clase raíz, la cual tiene siempre propiedades genéricas que serán heredadas por las clases hijas.

De esta manera, mediante las relaciones de herencia, objetos de distintas clases pueden tener atributos similares y exhibir comportamientos parecidos. La generalización y especialización expresan relaciones de inclusión entre conjuntos:

- La **generalización** consiste en crear una superclase más general, dada una o varias clases existentes.
- La **especialización** consiste en, dada una clase, generar una subclase de ésta que herede todo pero que implemente alguna funcionalidad extra que conceptualmente no debe ser añadida en la clase base.

Los atributos, métodos y relaciones de una clase se muestran en el nivel más alto de la jerarquía en el que son aplicables.

En la siguiente imagen se muestran ejemplos gráficos de generalización y especialización de clases.

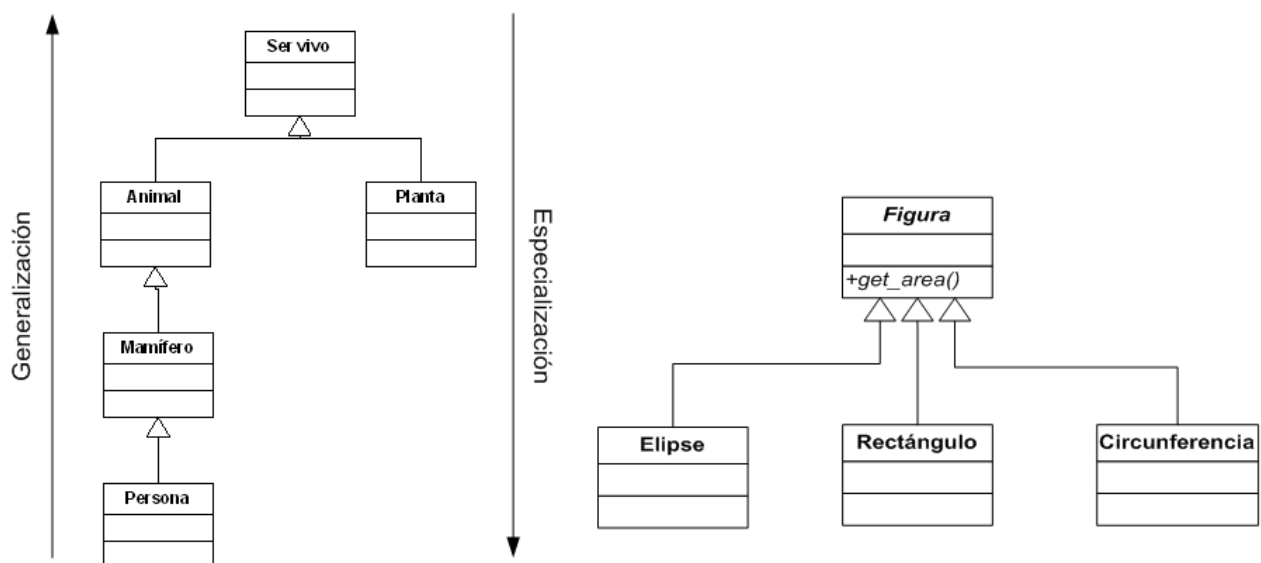


Figura 24 .- Relaciones de Generalización y Especialización derivadas del concepto de Herencia

Como se muestra en el ejemplo, la jerarquía puede incluso adquirir la forma de grafo en el caso de que exista la herencia múltiple, que consiste en que una clase hereda las propiedades de varias clases.

No obstante, cuando se implementan este tipo de relaciones en algunos lenguajes orientados a objetos como Java, que no permiten la herencia múltiple, se tiene que realizar por otros mecanismos, como es el uso de múltiples interfaces. Por el contrario, otros lenguajes como el lenguaje C++ sí lo permiten, siendo más potentes que Java en este sentido.

2.4.1.2 Criterios para identificar clases en el modelo de clases

Un problema fundamental que aparece ante cualquier persona que afronta el problema de diseñar una aplicación orientada a objetos es la identificación y elección de las clases apropiadas para la misma.

El diseño de clases es un tema de la Ingeniería del Software al que se le han dedicado diversos estudios y extensos libros. A modo general, las ideas bajo las cuales se debe crear una clase nueva son:

- ✓ Cuando aparezca un concepto nuevo, bien del problema a solucionar, bien de la solución del problema.
- ✓ Cada vez que se requiera un tipo de datos nuevo, se diseña una clase con unos atributos que modelen el rango del tipo de datos y unos métodos que modelen las operaciones soportadas.

A continuación, se explican algunos de los métodos propuestos, el análisis léxico y las tarjetas CRC.

Análisis léxico

Partiendo de un enunciado lo más preciso posible del problema, sobre él se estudian los nombres que aparecen. Los mismos constituyen posibles clases del modelo (parecen entidades/abstracciones existentes en el problema). Los verbos son operaciones candidatas de las clases con que aparecen o posibles relaciones de éstas con otras.

Tarjetas CRC

También conocido por los nombres de 'programación por contrato' o diseño dirigido por responsabilidades, intenta definir claramente las responsabilidades (qué información posee, qué operaciones se le pueden solicitar) de cada clase, así como cuáles son las otras clases a las que puede pedir colaboración para cumplir su misión (Collaboration Responsibilities Cards).

Mediante esta técnica pueden hacerse revisiones de completitud (¿falta algún colaborador?) y coherencia (¿hay alguna clase cuyo concepto no está bien definido?) del sistema, a la vez que van surgiendo, de forma natural, las operaciones e informaciones que ha de soportar una cierta clase.

Este método fue expuesto por Wirfs-Brock y adoptado por otros autores como B. Meyer.

2.5 Otras técnicas relacionadas

Adicionalmente a los modelos de dominio, diagramas de clases y de entidad relación (extendido) detallados en los apartados anteriores, existen otras técnicas en el conjunto de metodologías y técnicas de la ingeniería del software que, aunque menos usadas, también pueden resultar de utilidad durante la fase de análisis del dominio de un sistema de información. Es el caso de los diagramas de objetos, de paquetes, de estructura compuesta y el diccionario de datos.

2.5.1 Diagrama de objetos

Un diagrama de objetos se puede considerar un caso especial de un diagrama de clase. Los diagramas de objetos usan un subconjunto de elementos de un diagrama de clase para enfatizar la relación entre las instancias de las clases en algún punto en el tiempo. Estos son útiles para entender los diagramas de clases, pues muestran una “foto” del estado de un sistema en un punto en el tiempo determinado.

Sus elementos principales son:

- *El nombre de los objetos*: cada nombre es representado como un rectángulo, que contiene el nombre del objeto y su clase subrayadas y separadas por dos puntos.
- *Atributos*: como con las clases, los atributos se listan en un área inferior. La diferencia principal es que los atributos de los objetos deben tener un valor asignado.

El siguiente diagrama muestra las diferencias en apariencia entre un elemento clase y un elemento objeto. El elemento clase consiste en tres partes, divididas en compartimientos de nombres, atributos y operaciones; por defecto, los elementos objetos no tienen compartimientos. La forma en la que se muestran los nombres es también diferente: los nombres de los objetos están subrayados y pueden mostrar el nombre del clasificador desde el cual el objeto se instancia, mientras que en el caso de las clases no es así.

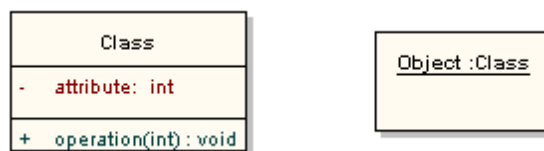


Figura 25 Clase vs Objeto

Además, un elemento puede tener cualquier número de atributos y operaciones, que se mostrarán en el objeto. Sin embargo, es posible definir el estado en tiempo de ejecución del objeto mostrando un conjunto de valores de atributos en la instancia particular, tal y como se muestra en la siguiente imagen:

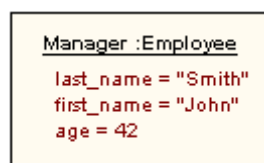


Figura 26 Objeto con atributos

En la siguiente figura se muestra un diagrama de objetos de ejemplo:

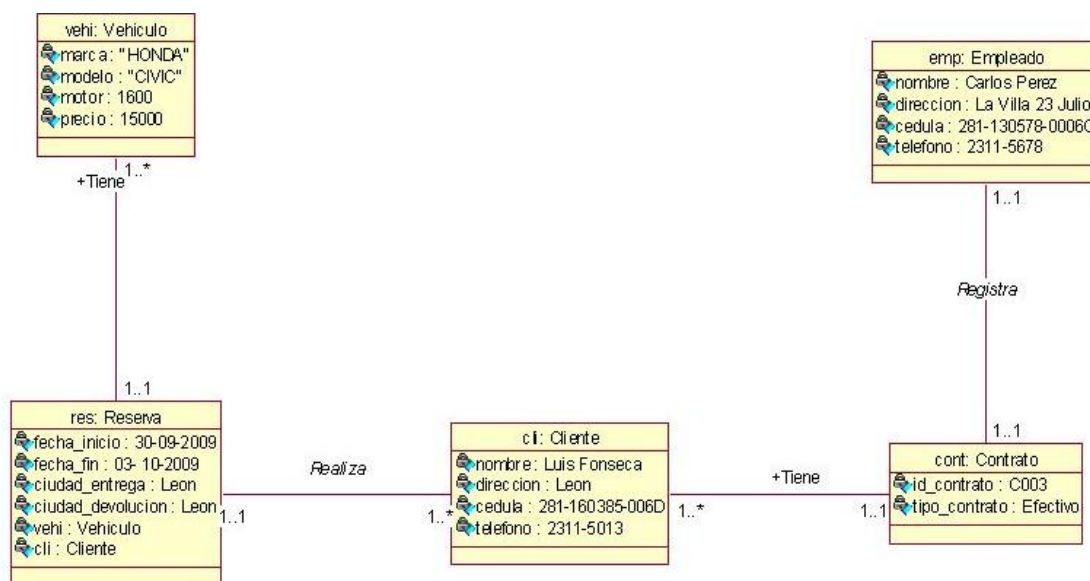


Figura 27 Ejemplo de diagrama de objetos

2.5.2 Diagrama de paquetes

Los diagramas de paquetes se usan para reflejar la organización de paquetes y sus elementos. El uso más común de los diagramas de paquetes son para organizar a su vez diagramas de casos de uso y diagramas de clases, a pesar de que su uso no está limitado exclusivamente a estos elementos UML, y puede ser utilizado como agrupación genérica de cualquier elemento de modelado como interfaces, componentes, nodos, colaboraciones, casos de uso e incluso otros paquetes. No obstante, dada la temática principal del presente tema, en este apartado se tomarán como referencia los diagramas de paquetes de clases.

La utilidad principal de un diagrama de paquetes es:

- Organizar los elementos en los modelos y facilitar su comprensión.
- Controlar el acceso a sus contenidos para controlar las líneas de separación de la arquitectura del sistema.

Los diagramas de paquetes son un mecanismo muy importante para gestionar la complejidad del modelado y son completamente diferentes a las clases. Mientras que las clases son abstracciones de aspectos del problema o la solución, los paquetes son elementos para organizar, es decir, no puede haber instancias de paquetes.

Cuando se usan para representaciones, los diagramas de paquete de los elementos de clase se usan para proveer una visualización de los espacios de nombres. En ese caso, los elementos contenidos en un paquete comparten el mismo espacio de nombres, y el hecho de compartir espacios de nombres requiere que los elementos contenidos en un espacio de nombres específico tienen nombres únicos que los identifican.

Los paquetes se pueden construir para representar relaciones tanto físicas como lógicas. Cuando se elige incluir las clases a los paquetes específicos, es útil asignar las clases con la misma jerarquía de herencia a los paquetes. Las clases que están relacionadas a través de la composición y las clases que colaboran que también son candidatas a ser incluidas en el mismo paquete.

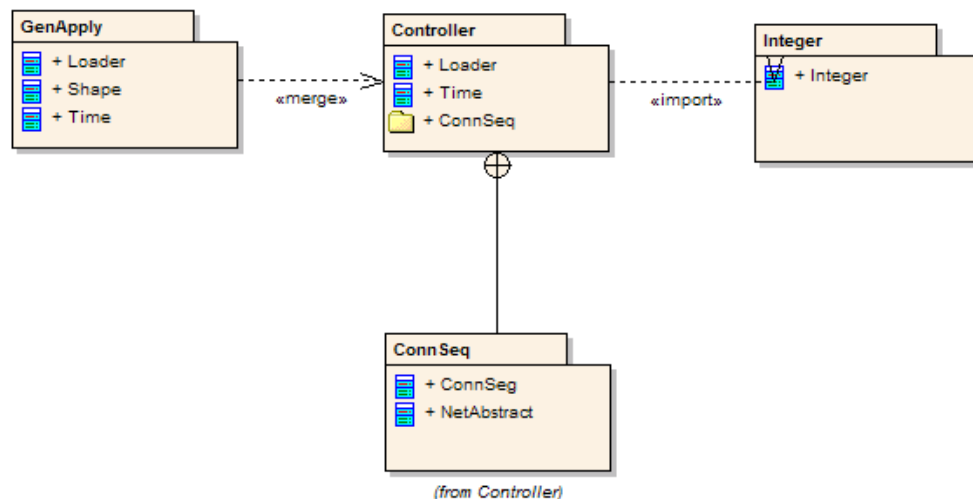


Figura 28 Diagrama de paquetes

Los paquetes se representan como carpetas y contienen los elementos que comparten un espacio de nombres; todos los elementos dentro de un paquete deben tener un identificador único. El paquete debe mostrar el nombre del paquete y puede, opcionalmente, mostrar los elementos dentro del paquete en compartimentos adicionales mediante las denominadas jerarquías.

Los paquetes bien diseñados agrupan elementos cercanos semánticamente, fuertemente cohesionados y débilmente acoplados. Entre un paquete y sus elementos existe una relación de composición, cada elemento del modelo pertenece a un único paquete (aunque puede ser referenciado desde otros) y si el paquete se destruye, todos los elementos que contiene son también destruidos.

Jerarquías de paquetes

Los paquetes pueden contener a otros paquetes formando jerarquías de paquetes. Los paquetes anidados tienen acceso al espacio de nombres del paquete que los contiene.

Dependencias entre paquetes

Las dependencias denotan que algún elemento de un paquete depende de los elementos en otro paquete. Puede haber de varios tipos:

- Combinación de paquetes o *merge*: la fuente de la combinación importa los contenidos importados y anidados del destino. Si existe un elemento dentro del origen y el destino, las definiciones del elemento origen se expandirán para incluir las definiciones del elemento contenidas en el destino. Todos los elementos agregados o actualizados por una combinación se notan por una relación de generalización desde el origen hasta el destino.
- Importación de paquetes o *import*: indica que los elementos dentro del paquete destino se importarán al paquete origen. El espacio de nombres del paquete origen ganará acceso a las clases destino; el espacio de nombre del destino no está afectado.
- Acceso o *access*: modelado con una dependencia estereotipada con «access» indica que el paquete origen tiene acceso a los elementos del paquete destino, sin que se añadan al espacio de nombres origen.
- Exportación de paquetes o *export*: esta dependencia es modelada implícitamente a través de la visibilidad pública de los elementos del paquete, de manera que cualquier otro paquete pueda usarlo.

2.5.3 Diagrama de estructura compuesta

Un diagrama de estructura compuesta es un diagrama que muestra la estructura interna de un elemento, incluyendo sus puntos de interacción a otras partes del sistema. Esto muestra la configuración y relación de las partes, que juntas realizan el comportamiento del clasificador contenido. En UML, un clasificador es un elemento del modelo que describe el comportamiento y características de la estructura, ejemplos de ellos son: clases, interfaces, casos de uso y actores.

Los elementos de clase han sido descritos en gran detalle en la sección en los diagramas de clase, por lo que en esta sección se describe la forma en que las clases se pueden mostrar como elementos compuestos exponiendo interfaces y conteniendo puertos y partes.

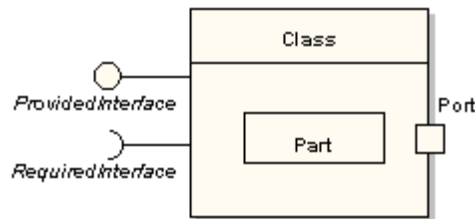


Figura 29 Diagrama de estructura compuesta

Parte

Una parte es un elemento que representa un conjunto de una o más instancias que pertenecen a una instancia del clasificador contenido. Por ejemplo, si una instancia de diagrama se apropia de un conjunto de elementos gráficos, luego los elementos gráficos se pueden representar como partes, si fuera útil hacer eso para modelar algún tipo de relación entre ellos. Hay que tener en cuenta que una parte se puede quitar de sus padres antes de que el padre se elimine, para que la parte no se elimine al mismo tiempo.

Una parte se muestra como un rectángulo no adornado dentro del cuerpo de una clase o del elemento componente, tal y como se puede ver en la figura anterior.

Puerto

Un puerto es un elemento escrito que representa una parte visible externa de una instancia del clasificador contenido. Los puertos definen la interacción entre un clasificador y su entorno. Un puerto puede aparecer en el límite de la parte contenida, una clase o una estructura compuesta. Un puerto puede especificar los servicios que un clasificador provee, así como también los servicios que éste requiere de su entorno.

Un puerto se muestra como un rectángulo nombrado en el borde del límite del clasificador al que pertenece.

Interfaces

Una interfaz es similar a una clase pero con un número de restricciones. Todas las operaciones de la interfaz son públicas y abstractas, y no proveen ninguna implementación predeterminada. Todos los atributos de la interfaz deben ser constantes. Sin embargo, mientras que una clase solamente puede heredar de una sola súper-clase, puede implementar interfaces múltiples.

Una interfaz, cuando está sola en un diagrama, se muestra como un rectángulo del elemento clase con la clave «*interface*» y con su nombre en *itálica* para denotar que es abstracto, o se muestra como un círculo.

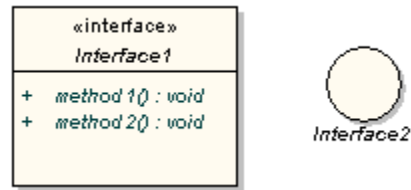


Figura 30 Interfaz

Hay que tener en cuenta que la notación del círculo no permite mostrar las operaciones de la interfaz.

Delegación

Un conector de *delegación* se usa para definir los trabajos internos de los puertos e interfaces externas del componente. Un conector delegar se muestra como una flecha con un estereotipo «delegate». Esto conecta un contrato externo de un componente, como se muestra por sus puertos, a la realización interna del comportamiento de la parte del componente.

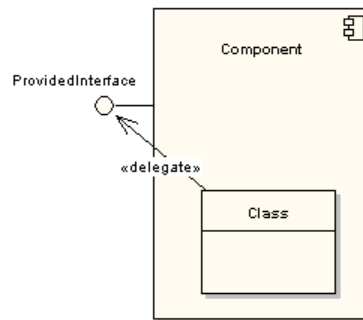


Figura 31 Delegación

Colaboración

Una *colaboración* define un conjunto de roles cooperativos usados colectivamente para ilustrar una funcionalidad específica. Una colaboración debería solamente mostrar los roles y los atributos requeridos para lograr sus tareas o funciones definidas. Aislar los roles primarios es un ejercicio de simplificar la estructura y clasificar el comportamiento, y permite su reutilización. Un elemento colaboración a menudo implementa un patrón. Un elemento colaboración se muestra como una elipse.

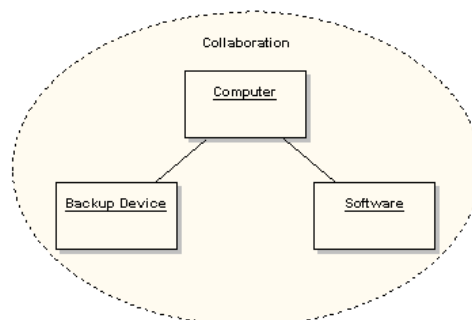


Figura 32 Colaboración

Enlace de Roles

Un conector *enlace de roles* se dibuja desde una colaboración a un clasificador que completa el rol. Esto se muestra como una línea de trazos con una punta de flecha y el estereotipo «enlace de roles».

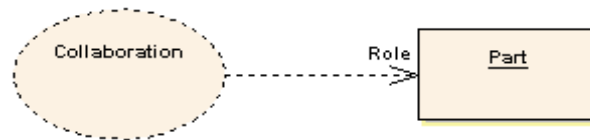


Figura 33 Ocurrencia

Representa

Un conector *representa* se puede dibujar desde una colaboración a un clasificador para mostrar que una colaboración se usa en el clasificador. Se muestra como una línea de trazos con una punta de flecha y el estereotipo «represents».

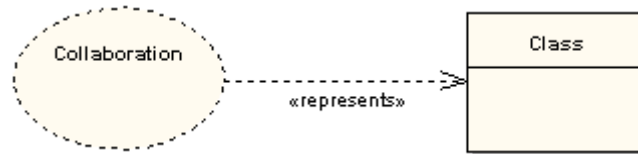


Figura 34 Representa

Ocurrencia

Un conector *ocurrencia* se puede dibujar desde una colaboración a un clasificador para mostrar que la colaboración representa el clasificador. Esto se muestra como una línea de trazos y el estereotipo «occurrence».

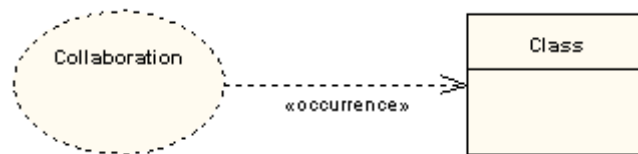


Figura 35 Ocurrencia

En el mundo real, el mundo de los objetos, los objetos están formados por más objetos. UML nos permite modelar dicha información por medio de relaciones de composición entre los objetos contenedores y sus partes.

Finalmente, cabe mencionar que un uso adicional que se puede dar a los diagramas de estructura compuesta es para mostrar las partes que colaboran, por ejemplo, en un caso de uso.

2.5.4 Diccionario de datos

El Diccionario de Datos es una técnica de análisis estructurado que permite especificar el significado y la composición de los datos. Un diccionario de datos contiene las descripciones de información que se usan en un cierto entorno (proyecto, base de datos, etc.).

La creación del diccionario de datos se va haciendo al mismo tiempo que se van confeccionando los restantes modelos del sistema, como el DFD (Diagrama de Flujo de Datos), etc., detallados en el siguiente tema.

Cuando se identifica un elemento en un diagrama se busca en el diccionario de datos. Si ya existe se usa, pero si no existe se procede a definirlo en el diccionario de datos antes de usarlo.

Hay que distinguir dos conceptos diferentes:

- Diccionario de Datos (DD): Es el conjunto de todas las entradas del diccionario de datos.
- Entrada del Diccionario de Datos (DDE): Es cada una de las entradas que contiene el diccionario con una definición de un dato.

2.5.4.1 Características del Diccionario de Datos

Las características que presentan los diccionarios de datos son:

- Los datos del diccionario se definen una sola vez, aun cuando se usen en varios sitios.
Esto permite eliminar la duplicidad de definiciones de datos y simplificar el uso de las definiciones, al no tener que rescribirlas una y otra vez.
- Toda definición de un dato debe describir cómo se compone ese dato y el conjunto de valores que éste puede tomar.

Mediante una notación estándar se define los elementos que componen el dato. Cada uno de estos elementos puede a su vez ser un elemento compuesto, el cual está compuesto por otros elementos, o un elemento simple. Así, por ejemplo, la entrada de diccionario “Dirección” se podría definir como:

Dirección ::= Tipo Vía + Nombre Vía + Número + Escalera + Piso + Puerta

- Un diccionario es útil cuando es la única fuente de información que se usa.
Toda la información del proyecto debe registrarse en el diccionario de datos, pues de lo contrario, las propiedades que tiene se pierden, al haber elementos en el proyecto que no están definidos en el diccionario de datos, o peor, están definidos de diferente forma.

2.5.4.2 Datos que se almacenan en un Diccionario de Datos

Aunque en un diccionario de datos se puede almacenar la información que se requiera para un proyecto en particular, para la mayor parte de los proyectos se almacena al menos la siguiente información.

- Nombre.
- Alias. Son nombres alternativos.
- Formato del Dato. Composición y valores del dato. Se usa una notación estándar para definirlo.
- Información Adicional. Cualquier información adicional que se considere y que pueda aportar a la definición de dicha información.
- Usuario y fecha de creación.
- Usuario y fecha de última modificación.
- Tipo de elemento.

Muchos de los diccionarios de datos son configurables y permiten añadir nuevos campos si así se requiere.

2.5.4.3 Definición del formato de los datos en el Diccionario de Datos

Los diccionarios de datos utilizan una notación estándar para definir cada uno de los elementos que contiene y cómo se relaciona con el resto de elementos del mismo. De esta forma, todos los desarrolladores usan el mismo formato y, además, se pueden realizar operaciones de comprobación y búsqueda sobre el diccionario. Esta definición del dato se almacena en el campo Formato del Dato que se ha visto en el apartado anterior.

Una de las notaciones que más se usa en la notación BNF (Backus-Naur Form). Siguiendo dicha notación, cada entrada del diccionario de datos tiene la siguiente estructura:

<Nombre_Dato> ::= <Componentes_de_los_Datos>

Cada entrada está formada por una parte izquierda, el operador “::=” y una parte derecha. La parte izquierda, al igual que en cualquier lenguaje de programación contiene el elemento a definir y la parte derecha la definición del elemento.

Nombre_Dato es el nombre que se le da al dato. Ejemplos de nombres pueden ser Dirección, Apellido, Número de Accesos, etc.

Componentes_de_los_Datos hace referencia a los diferentes elementos que conforman el dato. Puede haber uno o más elementos que conformen el dato, estando éstos separados por operadores. Cada uno de los componentes que conforman el dato puede ser de uno de los tipos siguientes:

- **Simple**, elementales o también llamados primitivos: Son tipos de datos atómicos y no pueden descomponerse en otros. Son simples, por ejemplo, los tipos de datos: entero, carácter, cadena de caracteres, número en coma flotante, etc.
- **Compuestos** o agregados: Son datos que están formados por otros tipos de datos más simples. Algunos de estos datos que conforman el dato compuesto pueden ser simples o compuestos.

Para la composición de los diferentes elementos que conforman el dato se utilizan los siguientes operadores:

- **Secuencia** (+). Permite especificar que un dato está compuesto por la secuencia de otros datos.

Código_Producto::= Area_Producto + Numero_Secuencial

- **Selección** ([... | ...]). Permite especificar la selección de un dato de entre un conjunto de ellos especificados entre paréntesis. Cada uno de los elementos se separa del siguiente por el símbolo “|”.

Color_Producto::=[rojo | verde | azul | marrón]

- **Iteración** (n{ }m). Permite especificar una repetición entre n y m veces del dato especificado entre llaves. En el ejemplo se muestra Productos_Adquiridos se define como una serie de elementos del tipo Producto. Este tipo producto se puede repetir entre 1 y m veces. Si no se conoce el límite inferior o el superior se deja en blanco. En el ejemplo, el número de productos que se pueden adquirir debe ser de al menos uno, pero no se especifica el número máximo de ellos que se pueden adquirir.

Productos_Adquiridos::= 1{Producto}

- **Opción** (()). El dato puede aparecer de forma opcional. En el ejemplo, como el prefijo está entre paréntesis, éste es opcional, mientras que los otros dos deben siempre aparecer.

Teléfono::= (Código_Pais) + Prefijo_Nacional + Número_Abonado

- **Comentario** (*Comentario*). Los comentarios permiten clarificar las definiciones del diccionario de datos.
- **Identificador** (@). Señala un campo o conjunto de campos que identifican cada ocurrencia de un almacén.

Código_Producto::= @Area_Producto + @Numero_Secuencial

- **Alias** (No es recomendable usarlo, pues se pueden producir inconsistencias). En el ejemplo se asigna el mismo formato al código del proveedor que el del suministrador.

Código_Proveedor::= Código_Suministrador

Las entradas del diccionario tienen un significado diferente cuando se define un tipo simple o compuesto. Cuando se define un elemento simple, los valores que hay en la parte derecha de la definición son el rango de valores que admite. Sin embargo, cuando son elementos compuestos, lo que se define en la parte derecha son los elementos a partir de los que se construye el elemento.

Así, en el siguiente ejemplo, el elemento simple Color_Producto tiene definidos a la derecha los valores que puede adoptar Color_Producto. Sin embargo, en el caso de Características_Producto se puede ver que está formado por los tres elementos que definen las características.

Color_Producto ::= [rojo | verde | azul | marrón]

Características_Producto ::= Color_Producto + Dimensiones_Producto + Modelo_Producto

Un diccionario de datos también permite definir una serie de características para cada entrada, como pueden ser:

- Valores relevantes.
- Valores por defecto.
- Unidades de medida en que se mide el elemento.
- Rango de valores que admite.

En el siguiente ejemplo se muestra el uso de estas características:

- Fecha Pedido::= * Fecha en la que se realizó el pedido *
Valor por Defecto: Fecha del Sistema
- Fecha Entrega::= * Fecha prevista de entrega del producto *
Valor por Defecto: Fecha del Sistema + 7 días hábiles
Rango de valores: [Fecha del Sistema .. Fecha del Sistema + 7 días hábiles]
- Peso del Producto::= * Peso del Producto Pedido *
Valor por Defecto: 2
Unidades : Kilogramos
Rango Valores : [1..50]

2.5.4.4 Operaciones que permite un Diccionario de Datos

El diccionario de datos facilita a los analistas las siguientes tareas:

- Localizar todos los lugares en los que se hace uso de un determinado dato.
- Determinar si un determinado dato está siendo o no utilizado.
- Definición exacta del significado de un dato. De esta forma se evitan duplicidades y malentendidos. Permite la comunicación entre analistas ya que la definición que uno hace de un dato es visible por el resto.
- Listado de todos los elementos que hay en el diccionario.
- Describe las relaciones entre los objetos que forman parte del modelo conceptual de datos.
- En las fases de comprobación de consistencia, el diccionario de datos permite hacer cruces con otros modelos para comprobar la consistencia y completitud de éstos.

2.5.4.5 Tipos de diccionarios

Existen dos tipos de diccionarios, en función de si éstos fuerzan o no a los programadores a usar en los programas las definiciones del diccionario de datos.

- **Diccionarios Activos.** Fuerzan a los programadores a utilizar las definiciones del diccionario. Estos diccionarios están conectados con el compilador para buscar la definición de los datos y si se modifica un dato forzar a recompilar.
- **Diccionarios Pasivos.** Permiten que los programadores puedan definir datos dentro de sus programas y que éstos no estén en el diccionario de datos.

3 Conclusión

Como se ha visto, el análisis de dominio incluye una serie de técnicas y herramientas que facilitan considerablemente el proceso de análisis de un sistema de información, especialmente desde la captura de la información y de las necesidades en el dominio del problema. Así, el análisis de dominio fue popularizado gracias a las iniciativas dirigidas a la reutilización del software, entendiendo como tal el proceso intelectual dedicado a abstraer las características vocabulario y reglas comunes de sistemas informáticos de un mismo tipo. De esta manera, se pretendía obtener las características básicas de los sistemas de información destinados a un mismo sector, actividad o proceso.

Adicionalmente, las técnicas de extracción y representación del conocimiento, pertenecientes al análisis de dominio, han sido utilizadas intensivamente por sectores específicos de tratamiento automático de textos y minería de textos, así como en procesos de diseño de ontologías.

Actualmente, las técnicas de análisis de dominio son ampliamente utilizadas dentro de la ingeniería del software para todo tipo de sistemas de información, y constituyen un pilar fundamental en la fase de Análisis del Sistema de Información.

4 Glosario

CASE: Computer Aided Software Engineering

OMG: Object Management Group

OO: Orientación a Objetos

DD: Diccionario de Datos.

DDE: Entrada del Diccionario de Datos.

UML: Unified Modelling Language.

CRC: Collaboration Responsibiliites Cards.

DFD: Diagrama de Flujo de Datos.

SGBD: Sistema de Gestión de Bases de Datos.

DED: Diagrama de Estructura de Datos.

SQL: Structured Query Language.

ER: Modelo Entidad Relación.

ASI: Análisis del Sistema de Información

ERS: Especificación de Requisitos Software

ACM Association of Computing Machinery

BNF Backus-Naur Form

RESUMEN ESQUEMÁTICO

- **INTRODUCCIÓN:** El análisis del dominio tiene como finalidad principal la captura y adquisición de la información que deben almacenar y procesar las aplicaciones informáticas orientadas a una función o uso.
- **ANÁLISIS DEL DOMINIO DE SISTEMAS**
 - Análisis y modelado de sistemas de información.
 - Durante el análisis de un sistema se estudia una necesidad existente, para proponer nuevos sistemas o actualizar uno existente. Se refleja en el documento ERS, la Especificación de Requisitos Software.
 - Posteriormente se realiza el modelado del sistema, que sirve de base para el diseño del sistema. Existen dos tipos de modelado:
 - Estructurado: DFD, DD, E/R, DED.
 - Orientado a objetos: clases, casos de uso, dominio.
 - Modelado de dominio: dominio es una representación visual de clases conceptuales o de objetos reales en un dominio de interés y las relaciones presentes entre sí.
 - Se representa con el modelo de clases.
 - Guía de construcción:
 - Listar clases conceptuales candidatas.
 - Definir nombre para cada clase.
 - Añadir asociaciones.
 - Definir atributos.
 - Definir especializaciones.
 - Modelo E/R: conjunto de conceptos, reglas y convenciones que permiten describir los datos de una parcela del mundo real.
 - Propuesto por Peter Chen en 1976.
 - Elementos: entidades, relaciones regulares o débiles, atributos (monovalorados/multivalorados, simples/compuestos), restricciones (cardinalidad y tipo de correspondencia, semánticas estructurales y de usuario).
 - Extendido: añade especialización-generalización, cardinalidad mínima, dependencia en existencia e identificación, agregación.
 - Traducción al modelo relacional: hay que realizar análisis de redundancias, eliminar generalidades, transformar entidades fuertes, transformar las relaciones a tablas, transformar las entidades débiles.
 - Ventajas: expresa con mucha precisión el esquema conceptual, facilita comunicación.
 - Desventajas: carece de soporte formal y SGBD no lo implementan.
 - Modelo de clases: es un diagrama de estructura de UML que pretende mostrar la estructura interna del sistema diseñado al nivel de clases e interfaces, con sus características, restricciones y relaciones, asociaciones, generalizaciones...etc.
 - Elementos: clases, variables o atributos de instancia, relaciones entre clases, clases abstractas, objetos, mensajes y métodos.

- Otras técnicas relacionadas:
 - Diagrama de objetos: objetos usan un subconjunto de elementos de un diagrama de clase para enfatizar la relación entre las instancias de las clases en algún punto en el tiempo.
 - Elementos: objetos y atributos.
 - Diagrama de paquetes: son para organizar a su vez diagramas de casos de uso y diagramas de clases, a pesar de que el uso de los diagramas de paquete no es limitado a estos elementos UML y puede ser utilizado como agrupación genérica de cualquier elemento de modelado.
 - Elementos: paquetes, jerarquías y dependencias.
 - Diagrama de estructura compuesta: muestra la estructura interna de un clasificador, incluyendo sus puntos de interacción a otras partes del sistema.
 - Elementos: parte, puerto, interfaces, delegación, colaboración, enlace de roles, representa, ocurrencia.
 - Diccionario de datos: una técnica de análisis estructurado que permite especificar el significado y la composición de los datos.
 - Notación BNF (Backus-Naur Form)
 - Tipos activos y pasivos.
- **CONCLUSIÓN:** el análisis de dominio fue popularizado gracias a las iniciativas dirigidas a la reutilización del software, mientras que las técnicas de extracción y representación del conocimiento, utilizadas en el análisis de dominio, han sido utilizadas intensivamente por sectores específicos de tratamiento automático de textos y minería de textos.

PREGUNTAS DE TEST

- 1) **Genéricamente, la encapsulación de un dato y su comportamiento constituye:**
 - a) Una identidad.
 - b) La encapsulación.
 - c) Un objeto.
 - d) Un método.

- 2) **En Orientación a Objetos, una solicitud que pide a un objeto que se comporte de alguna forma determinada, recibe el nombre de:**
 - a) Método.
 - b) Mensaje.
 - c) Proceso.
 - d) Procedimiento.

- 3) **Se entiende por “protocolo del objeto”:**
 - a) El conjunto de instrucciones que gestionan los datos del objeto.
 - b) El conjunto de atributos del objeto.
 - c) El conjunto de mensajes al que puede responder el objeto
 - d) El conjunto de identificadores del objeto.

- 4) **El conjunto de instrucciones que gestionan los datos de una clase, se denomina:**
 - a) Método.
 - b) Mensaje.
 - c) Protocolo de la clase.
 - d) Ninguna de las anteriores.

- 5) **Para expresar que los datos relativos a algún objeto están almacenados junto con el proceso que crea y manipula esos datos, se utiliza el término:**
 - a) Persistencia.
 - b) Identidad.
 - c) Herencia.
 - d) Encapsulación.

- 6) **En el paradigma de la orientación a objetos, la herencia:**
 - a) Permite derivar clases nuevas a partir de otras ya definidas.
 - b) Puede ser del tipo “usa” o del tipo “es un”.
 - c) Se clasifica en herencia de clases, de objetos, de datos y de métodos.
 - d) Es útil para que unos métodos utilicen otros.

- 7) **En los sistemas Orientados a Objetos, la agregación es una relación:**
 - a) Entre clases, definida por la expresión “es parte de”.
 - b) Entre clases, definida por la expresión “es un”.
 - c) Entre una clase y un objeto.
 - d) Entre objetos.

- 8) **El principio que establece que los atributos de una clase no sean accesibles desde el exterior, sino a través del intercambio de mensajes establecido, se denomina:**

- a) Encapsulación.
- b) Identidad.
- c) Herencia.
- d) Polimorfismo.

9) La propiedad por la cual un mismo mensaje puede originar conductas diferentes al ser recibido por objetos diferentes, se denomina:

- a) Reusabilidad.
- b) Herencia.
- c) Extensibilidad.
- d) Polimorfismo.

10) La relación “un perro es un mamífero” es una relación de:

- a) Agregación.
- b) Generalización.
- c) Partición.
- d) Exclusividad.

11) El que una clase pueda adquirir las propiedades de otra clase jerárquicamente superior de debe a la propiedad de:

- a) Polimorfismo.
- b) Herencia.
- c) Extensibilidad
- d) Nunca puede darse el hecho planteado.

12) ¿Qué afirmación de las siguientes no es correcta?

- a) Las descripciones de las clases incluyen la definición de los atributos, la descripción de la interfaz de clase y el conjunto de transacciones válidas entre los posibles estados de una instancia de clase.
- b) La característica de extensibilidad de los diseños orientados a objetos se debe a la relación de herencia y polimorfismo.
- c) Las clases se diseñan de forma que presenten un acoplamiento fuerte y una cohesión débil.
- d) La separación de la interfaz de clase de su implementación permite la ocultación de detalles de implementación y, por tanto, que la misma especificación de interfaz pueda implementarse de diversas maneras, de forma transparente a las otras clases clientes de la misma.

13) Señalar la respuesta correcta:

- a) En el proceso de identificación de clases en el diseño orientado a objetos, una técnica utilizada es la del análisis léxico de las especificaciones del sistema, donde los verbos son candidatos a clases y los nombres son candidatos a operaciones.
- b) Una característica del diseño orientado a objetos es el fuerte acoplamiento entre las clases ya que una clase puede acceder a todas las operaciones y datos que contenga otra clase.
- c) Las clases constituyen uno de los principales mecanismos de abstracción en el diseño orientado a objetos.
- d) Una clase madre hereda implícitamente todos los atributos y operaciones de la clase hija.

14) El modelo Entidad Relación de Chen es:

- a) Una técnica para la modelización semántica de datos.
- b) Una herramienta automatizada de análisis de requisitos.
- c) Una técnica para el modelado de procesos.

- d) Una metodología para el desarrollo de sistemas de información.

15) En el modelo Entidad/ Relación, ¿qué es el tipo de correspondencia?

- a) Es el número de entidades que participan en una asociación.
- b) Representa el tipo de entidades que participan en una asociación
- c) Es el número mínimo y máximo de ocurrencias de una entidad que pueden estar asociadas con una ocurrencia de otra u otras entidades participantes en la asociación.
- d) Es el número máximo de ocurrencias de cada entidad conectada que pueden intervenir en una ocurrencia de la asociación.

16) En el modelo E/R, las entidades débiles:

- a) Dependen de la existencia de otro tipo de entidad.
- b) Solamente pueden tener un único atributo de clave propio.
- c) Tienen atributos de clave propios, pero coinciden con los de la entidad fuerte de la que dependen.
- d) Se representan mediante un rectángulo de trazo discontinuo.

17) Cuando se transforma un modelo E/R a modelo relacional las relaciones 1:n:

- a) Se transforman por el mecanismo de propagación de clave ajena.
- b) Generan una tabla igual que una relación n:m.
- c) Dependiendo del tipo de consultas posteriores se genera o no la tabla.
- d) Si el volumen de datos es muy grande se genera la tabla.

SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
1	c
2	b
3	c
4	a
5	d
6	a
7	a
8	a
9	d
10	b
11	b
12	c
13	c
14	a
15	d
16	a
17	a

BIBLIOGRAFÍA BÁSICA

1. *Fundamentos de Sistemas de Bases de Datos*, 5ª edición, 2011 (Capítulo 3). Elmasri y Navathe
2. *Database systems: the complete book*. Prentice-Hall. Garcia-Molina, Ullman y Widom
3. Algunas extensiones del modelo E/R orientadas a objetos. Miguel Adoración, Piattini, Mario
4. *Metodología MÉTRICA Versión 3*.
5. *Ingeniería del Software. Un Enfoque Práctico*. Ed. McGraw Hill. 7ª Ed. 2014. Pressman, Roger. S.
6. *Ingeniería del Software*. Ed. Pearson. 9ª Ed. 2012. Sommerville, Ian.
7. *Object-Oriented Analysis and Design*, Grady Booch, Addison-Wesley.
8. *Programación orientada a objetos*. 2ª Ed. Prentice-Hall, 1998. Joyanes, L.
9. *Construcción de software orientado a objetos*. 2ª Ed. Prentice-Hall, 1999. Meyer, B.
10. Diagramas de paquetes
http://www.sparxsystems.com.ar/resources/tutorial/uml2_packagediagram.html
11. Diagrama de estructura compuesta
http://www.sparxsystems.com.ar/resources/tutorial/uml2_compositediagram.html
12. Modelado de dominio http://www-2.dc.uba.ar/materias/isoft1/is1-2005_2/apuntes/SlidesDC.pdf
13. Diagrama de dominio https://dosideas.com/wiki/Diagrama_de_dominio
14. Técnicas de análisis de dominio: organización del conocimiento para la construcción de sistemas software. Ricardo Eito Brun.
15. Relaciones entre clases: Diagramas de clases UML. <http://elvex.ugr.es/decsai/java/pdf/3C-Relaciones.pdf>

BIBLIOGRAFÍA PARA AMPLIAR EL TEMA

1. Modelo ER. Jose Ramón Paramá Gabia. <http://docencia.lbd.udc.es/bdd-grao/teoria/tema2/2.2.-ElModeloER.pdf>
2. Arquitectura Lógica del Sistema en desarrollo OO
<http://www.istr.unican.es/asignaturas/is1/is1-t11-trans.pdf>
3. Especificación OMG UML 2.5 <http://www.omg.org/spec/UML/2.5/PDF/>
4. UM 2.5 Iniciación, ejemplos y ejercicios corregidos. Laurent Debrauwer, Fien Van Der Heyde.